



Отчет о проверке

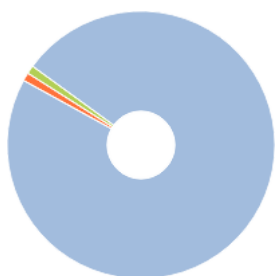
Автор: Острожных Д.

Название документа: Цифровая платформа оптовых закупок товаров для организаций

Проверяющий: Жеман Олег Вячеславович

Организация: Государственное учреждение «Национальная библиотека Беларуси»

РЕЗУЛЬТАТЫ ПРОВЕРКИ



Совпадения:
1,46%



Оригинальность:
98,03%



ИИ-контент:
19,52%



Цитирования:
0,51%



Самоцитирования:
0%



«Совпадения», «Цитирования», «Самоцитирования», «Оригинальность» являются отдельными показателями, отображаются в процентах и в сумме дают 100%, что соответствует проверенному тексту документа.



Есть подозрения на следующие группы маскировки заимствований: Сгенерированный текст на страницах: 11, 13, 18, 19, 20, 21, 22, 23, 26, 29... еще на 18 стр.



Проверено: 86,8% текста документа, исключено из проверки: 13,2% текста документа. Разделы и элементы, отключенные пользователем: Титульный лист, Содержание, Библиография, Приложение

- Совпадения** — фрагменты проверяемого текста, полностью или частично сходные с найденными источниками, за исключением фрагментов, которые система отнесла к цитированию или самоцитированию. Показатель «Совпадения» — это доля фрагментов проверяемого текста, отнесенных к совпадениям, в общем объеме текста.
- Самоцитирование** — фрагменты проверяемого текста, совпадающие или почти совпадающие с фрагментом текста источника, автором или соавтором которого является автор проверяемого документа. Показатель «Самоцитирования» — это доля фрагментов текста, отнесенных к самоцитированию, в общем объеме текста.
- Цитирование** — фрагменты проверяемого текста, которые не являются авторскими, но которые система отнесла к корректно оформленным. К цитированиям относятся также шаблонные фразы; библиография; фрагменты текста, найденные модулем поиска «СПС Гарант: нормативно-правовая документация». Показатель «Цитирования» — это доля фрагментов проверяемого текста, отнесенных к цитированию, в общем объеме текста.
- Текстовое пересечение** — фрагмент текста проверяемого документа, совпадающий или почти совпадающий с фрагментом текста источника.
- Источник** — документ, проиндексированный в системе и содержащийся в модуле поиска, по которому проводится проверка.
- Оригинальный текст** — фрагменты проверяемого текста, не обнаруженные ни в одном источнике и не отмеченные ни одним из модулей поиска. Показатель «Оригинальность» — это доля фрагментов проверяемого текста, отнесенных к оригинальному тексту, в общем объеме текста.

Обращаем Ваше внимание, что система находит текстовые совпадения проверяемого документа с проиндексированными в системе источниками. При этом система является вспомогательным инструментом, определение корректности и правомерности совпадений или цитирований, а также авторства текстовых фрагментов проверяемого документа остается в компетенции проверяющего.

ИНФОРМАЦИЯ О ДОКУМЕНТЕ

Номер документа: 3922

Тип документа: Дипломная работа

Дата проверки: 07.05.2026 17:00:26

Дата корректировки: Нет

Количество страниц: 80

Символов в тексте: 132729

Слов в тексте: 15731

Число предложений: 5314

Комментарий: не указано

ПАРАМЕТРЫ ПРОВЕРКИ

Выполнена проверка с учетом редактирования: Да

Выполнено распознавание текста (OCR): Нет

Выполнена проверка с учетом структуры: Да

Разделы и элементы, отключенные пользователем: Титульный лист, Содержание, Библиография, Приложение

Модули поиска: Цитирование, СМИ России и СНГ, Перефразированные заимствования по коллекции Интернет в английском сегменте, Профессиональная лексика. Медицина, IEEE, Переводные заимствования, Профессиональная лексика. АПК и биотех, PubMed, Шаблонные фразы, Сводная коллекция научных работ Беларуси, Патенты СССР, РФ, СНГ, Коллекция НБУ, ИПС Адилет, Публикации РГБ, Публикации eLIBRARY, Сводная коллекция ЭБС, Медицина, Переводные заимствования по базе публикаций открытого доступа PubMed, Перефразирования по базе публикаций открытого доступа PubMed, СПС ГАРАНТ: аналитика, Перефразирования по Коллекции открытых публикаций международных издательств, Коллекция открытых публикаций международных издательств, СПС ГАРАНТ: нормативно-правовая документация, Кольцо вузов, Переводные заимствования IEEE, Переводные заимствования по Коллекции открытых публикаций международных издательств, Перефразирования по коллекции IEEE, Переводные заимствования по коллекции Интернет в английском сегменте, Публикации РГБ (переводы и перефразирования), Публикации eLIBRARY (переводы и перефразирования), Перефразирования по СПС ГАРАНТ: аналитика, Переводные заимствования по коллекции Гарант: аналитика, Перефразированные заимствования по коллекции Интернет в русском сегменте, Переводные заимствования по коллекции Интернет в русском сегменте, Интернет Плюс, Собственная коллекция компании

ИСТОЧНИКИ

№	Доля в тексте	Доля в отчете	Источник	Актуален на	Модуль поиска	Комментарий
[01]	0,62%	0,62%	https://pure.tue.nl/ws/portalfiles/p... https://pure.tue.nl	02 Янв 2023	Переводные заимствования по коллекции Интернет в английском сегменте	
[02]	0,51%	0%	Нормативное обеспечение инфо... https://book.ru	01 Янв 2020	Сводная коллекция ЭБС	
[03]	0,51%	0%	Нормативное обеспечение инфо... https://book.ru	01 Янв 2017	Сводная коллекция ЭБС	
[04]	0,51%	0,51%	Пояснительная записка к проект... http://ivo.garant.ru	26 Янв 2013	СПС ГАРАНТ: нормативно-правовая документация	
[05]	0,51%	0%	Приказ Департамента государств... http://ivo.garant.ru	22 Авг 2015	СПС ГАРАНТ: нормативно-правовая документация	
[06]	0,51%	0%	ПРИЛОЖЕНИЕ 4. Перечень норм... http://elibrary.ru	01 Янв 2013	Публикации eLIBRARY	Источник исключен. Причина: Маленький процент пересечения.
[07]	0,5%	0%	Автоматизация процесса направ...	25 Июн 2025	Кольцо вузов	
[08]	0,5%	0%	Диплом Лашин ЕВ	22 Июн 2025	Кольцо вузов	
[09]	0,5%	0%	Тынков_ВКР_Автоматизация_про...	28 Янв 2025	Кольцо вузов	
[10]	0,5%	0%	ВКР Лашин ЕВ	24 Июн 2025	Кольцо вузов	
[11]	0,49%	0%	Внедрение информационных сис... https://book.ru	01 Янв 2024	Сводная коллекция ЭБС	Источник исключен. Причина: Маленький процент пересечения.
[12]	0,48%	0%	не указано	13 Янв 2022	Шаблонные фразы	Источник исключен. Причина: Маленький процент пересечения.
[13]	0,47%	0%	Теория и практика бизнес-анали... https://geotar.ru	20 Апр 2022	Сводная коллекция ЭБС	
[14]	0,47%	0%	Руководство по использованию с... http://elibrary.ru	01 Янв 2018	Публикации eLIBRARY	
[15]	0,47%	0%	РАЗРАБОТКА ТЕХНИЧЕСКОГО ЗАД... https://scienceforum.ru	19 Мая 2019	Интернет Плюс	
[16]	0,47%	0%	РАЗРАБОТКА ТЕХНИЧЕСКОГО ЗАД... https://scienceforum.ru	27 Апр 2022	Интернет Плюс	
[17]	0,47%	0%	РАЗРАБОТКА ТЕХНИЧЕСКОГО ЗАД... https://scienceforum.ru	27 Апр 2021	Интернет Плюс	
[18]	0,47%	0%	РАЗРАБОТКА ТЕХНИЧЕСКОГО ЗАД... https://scienceforum.ru	28 Мая 2025	Интернет Плюс	
[19]	0,45%	0%	Правовые основы обеспечения и... http://journal.ib-bank.ru	19 Июн 2013	СМИ России и СНГ	

[20]	0,45%	0%	Проектирование единого инфор...	19 Дек 2016	Медицина	Источник исключен. Причина: Маленький процент пересечения.
[21]	0,44%	0%	Богданов, Александр Евгеньевич ... http://dlib.rsl.ru	01 Янв 2016	Публикации РГБ	
[22]	0,44%	0%	Ю. И. Коваленко Защита информ... http://dlib.rsl.ru	01 Янв 2016	Публикации РГБ	
[23]	0,44%	0%	Катастрофоустойчивость инфор... http://elibrary.ru	01 Янв 2020	Публикации eLIBRARY	
[24]	0,44%	0%	Коржов В.Ю., Великанов А.П., Ив... http://ivo.garant.ru	22 Авг 2015	СПС ГАРАНТ: аналитика	
[25]	0,44%	0%	ГОСТ Р 51189-98 Средства програ... https://normadocs.ru	21 Окт 2025	Интернет Плюс	
[26]	0,42%	0%	О жизненном цикле системы эле... http://ivo.garant.ru	23 Сен 2023	СПС ГАРАНТ: аналитика	
[27]	0,41%	0,26%	An Intelligent Academic Assistant w...	05 Янв 2026	ПЕРЕФРАЗИРОВАНИЯ ПО КОЛЛЕКЦИИ IEEE	
[28]	0,39%	0%	Перечень технической и техноло... http://ivo.garant.ru	24 Сен 2020	СПС ГАРАНТ: нормативно-правовая документация	
[29]	0,36%	0%	Национальный стандарт РФ ГОС... http://ivo.garant.ru	01 Ноя 2020	СПС ГАРАНТ: нормативно-правовая документация	
[30]	0,35%	0%	Песчанникова, Елена Николаевн... http://dlib.rsl.ru	01 Янв 2019	Публикации РГБ	Источник исключен. Причина: Маленький процент пересечения.
[31]	0,35%	0%	Разработка Технического задани... https://pcnews.ru	16 Авг 2019	СМИ России и СНГ	Источник исключен. Причина: Маленький процент пересечения.
[32]	0,35%	0%	ОПЫТ ОБУЧЕНИЯ СТУДЕНТОВ ТЕ... http://elibrary.ru	01 Янв 2024	Публикации eLIBRARY	
[33]	0,35%	0%	Modeling a Program with Vulnerabi... https://openalex.org	13 Мар 2023	Коллекция открытых публикаций международных издательств	Источник исключен. Причина: Маленький процент пересечения.
[34]	0,35%	0%	Кодлычевская София диплом	05 Июн 2024	Кольцо вузов	
[35]	0,32%	0%	XcoiebtYCs8q.pdf https://elib.pnzgu.ru	25 Июн 2025	Переводные заимствования по коллекции Интернет в русском сегменте	Источник исключен. Причина: Маленький процент пересечения.
[36]	0,31%	0,08%	Жданович, Анастасия Александр... http://dlib.rsl.ru	01 Янв 2010	Публикации РГБ	
[37]	0,31%	0%	Дубинкина, Любовь Владимиров... http://dlib.rsl.ru	01 Янв 2008	Публикации РГБ	
[38]	0,29%	0,29%	Веб-приложение для продажи ав...	18 Окт 2025	Публикации eLIBRARY (переводы и перефразирования)	
[39]	0,28%	0%	Леонов, Дмитрий Геннадьевич М... http://dlib.rsl.ru	18 Сен 2024	Публикации РГБ	
[40]	0,27%	0%	Стандартизация и разработка пр...	19 Дек 2016	Медицина	
[41]	0,23%	0%	Подобрий, Александр Николаеви... http://dlib.rsl.ru	01 Янв 2013	Публикации РГБ	
[42]	0,23%	0%	Основы организации труда в ци... http://ivo.garant.ru	25 Мар 2023	СПС ГАРАНТ: аналитика	
[43]	0,23%	0%	Магистерская диссертация_Кари...	10 Июн 2019	Кольцо вузов	
[44]	0,23%	0%	https://studfile.net/preview/926932... https://studfile.net	09 Дек 2019	Интернет Плюс	
[45]	0,22%	0%	Способ комплексной оптимизац... https://fips.ru	30 Мар 2026	Патенты СССР, РФ, СНГ	
[46]	0,22%	0,01%	Рустамов, Павел Анварович Инф... http://dlib.rsl.ru	16 Ноя 2025	Публикации РГБ (переводы и перефразирования)	
[47]	0,21%	0%	Инжиниринг объектов интеллект...	19 Дек 2016	Медицина	Источник исключен. Причина: Маленький процент пересечения.
[48]	0,21%	0%	В МЭСИ состоялось рабочее сове... http://prtime.ru	10 Сен 2013	СМИ России и СНГ	
[49]	0,21%	0%	В МЭСИ состоялось рабочее сове... http://moskva.bezformata.ru	10 Сен 2013	СМИ России и СНГ	
[50]	0,21%	0%	Национальный стандарт РФ ГОС... http://ivo.garant.ru	25 Сен 2018	СПС ГАРАНТ: нормативно-правовая документация	
[51]	0,21%	0%	Об утверждении плана-графика ... https://adilet.zan.kz	12 Дек 2025	ИПС Адилет	Источник исключен. Причина: Маленький процент пересечения.

[52]	0,21%	0%	О перечне стандартов и рекомен... https://adilet.zan.kz	12 Дек 2025	ИПС Адилет	Источник исключен. Причина: Маленький процент пересечения.
[53]	0,2%	0%	Технология проектирования авт...	20 Дек 2016	Медицина	Источник исключен. Причина: Маленький процент пересечения.
[54]	0,2%	0,2%	Муссель К.М. Платёжные техноло... http://ivo.garant.ru	21 Фев 2015	Перефразирования по СПС ГАРАНТ: аналитика	
[55]	0,19%	0%	Воронова, Ольга Владимировна ... http://dlib.rsl.ru	01 Янв 2025	Публикации РГБ (переводы и перефразирования)	Источник исключен. Причина: Маленький процент пересечения.
[56]	0,19%	0%	Коржов В.Ю., Великанов А.П., Ив... http://ivo.garant.ru	22 Авг 2015	Перефразирования по СПС ГАРАНТ: аналитика	Источник исключен. Причина: Маленький процент пересечения.
[57]	0,18%	0%	Автоматизация процесса малярн...	19 Июн 2017	Кольцо вузов	Источник исключен. Причина: Маленький процент пересечения.
[58]	0,17%	0%	Типовая форма ведомственной п... http://elibrary.ru	раньше 2011	Публикации eLIBRARY	Источник исключен. Причина: Маленький процент пересечения.
[59]	0,16%	0%	5360157.ГЛ компании Гарант - По... http://ivo.garant.ru	28 Фев 2025	СПС ГАРАНТ: аналитика	Источник исключен. Причина: Маленький процент пересечения.
[60]	0,16%	0%	http://downloads.v8.1c.ru/content/... http://downloads.v8.1c.ru	25 Фев 2025	Переводные заимствования по коллекции Интернет в русском сегменте	Источник исключен. Причина: Маленький процент пересечения.
[61]	0,15%	0%	Основы организации труда в ци... http://ivo.garant.ru	25 Мар 2023	Переводные заимствования по коллекции Гарант: аналитика	Источник исключен. Причина: Маленький процент пересечения.
[62]	0,15%	0%	Алексанов А.К., Доронин А.М., Де...	24 Июл 2025	Переводные заимствования по коллекции Гарант: аналитика	Источник исключен. Причина: Маленький процент пересечения.
[63]	0,14%	0%	Диплом.docx	15 Янв 2024	Кольцо вузов	Источник исключен. Причина: Маленький процент пересечения.
[64]	0,12%	0%	Мошковский Диплом	21 Мая 2025	Кольцо вузов	Источник исключен. Причина: Маленький процент пересечения.
[65]	0,12%	0%	Об утверждении технического за... https://adilet.zan.kz	15 Дек 2025	ИПС Адилет	Источник исключен. Причина: Маленький процент пересечения.
[66]	0,1%	0%	https://www.mitso.by/web/uploads... https://mitso.by	26 Янв 2022	Интернет Плюс	Источник исключен. Причина: Маленький процент пересечения.
[67]	0,09%	0%	Руководство по выполнению вып... http://biblioclub.ru	21 Янв 2020	Сводная коллекция ЭБС	Источник исключен. Причина: Маленький процент пересечения.

Учреждение образования Федерации профсоюзов Беларуси
«Международный университет «МИТСО»

Экономический факультет

Кафедра информационных технологий

Допущен к защите
заведующий кафедрой
_____ О.Б. Хорошко
« ____ » _____ 2026

ДИПЛОМНЫЙ ПРОЕКТ

Цифровая платформа оптовых закупок товаров для организаций

Автор: _____
Острожных Денис Михайлович
Курс 4, группа № 2221
Специальность: информационные
системы и технологии

Научный руководитель: _____
д-р экон. наук, доцент:
Гардейчик Сергей Михайлович

Нормоконтролер _____ Ловчая Вероника Игоревна

Минск 2026

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ ФЕДЕРАЦИИ ПРОФСОЮЗОВ БЕЛАРУСИ
«МЕЖДУНАРОДНЫЙ УНИВЕРСИТЕТ «МИТСО»

Экономический факультет

Специальность 1-40 05 01-02 «Информационные системы и технологии»

Кафедра информационных технологий

УТВЕРЖДАЮ

Заведующий кафедрой

информационных технологий

_____ О.Б.Хорошко

«__» _____ 2026 г

ЗАДАНИЕ

на дипломный проект

Обучающемуся Острожных Денис Михайлович

Курс 4 Учебная группа 2221

Специальность 1-40 05 01-02 «Информационные системы и технологии»

Тема дипломной работы: **Цифровая платформа оптовых закупок товаров для организаций**

Утверждена руководителем УВО от «20» марта 2026 г. № 87- К (ст)

Исходные данные к дипломному проекту:

- ГОСТ 34.003-90 Информационная технология. Комплекс стандартов на автоматизированные системы. Термины и определения;
- ГОСТ 34.602-89 Информационная технология. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы; 4
- РД 50-34.698-90 Автоматизированные системы. Требования к содержанию документов; 4
- ГОСТ Р ИСО/МЭК 12207-2010 «Информационная технология. Системная и программная инженерия. Процессы жизненного цикла программных средств»;
- ГОСТ Р ИСО/МЭК 15288-2005 Информационная технология. Системная инженерия. Процессы жизненного цикла систем; 4
- Гагарина, 46 Информационные системы и технологии : учебник / Л. Г. Гагарина, Т. А. Киселева. – 3-е изд. – М. : ИНФРА-М, 2024. – 384 с;
- Коннолли, Т. Базы данных: проектирование, реализация и сопровождение / Т. Коннолли, К. Бегг. – 6-е русскоязычное изд. – М. : Вильямс, 2025. – 1440 с;

- Кузнецов, С. Д. Базы данных : учебник / С. Д. Кузнецов. – 3-е изд., перераб. и доп. – М. : Академия, 2023. – 496 с;
- Фримен, Э. Паттерны проектирования / Э. Фримен, Б. Бейтс, К. Сьерра. – СПб. : Питер, 2022. – 928 с.

Перечень подлежащих разработке вопросов или краткое содержание расчетно-пояснительной записки:

Дипломный проект является самостоятельно выполненной проектно-конструкторской разработкой. Тема актуальна для предприятия ООО «Семь драконов», на котором проходила преддипломная практика. Проект содержит элемент новизны и практической значимости. Состоит из следующих взаимосвязанных частей.

Введения. Во введении отражена актуальность темы: компания ООО «Семь драконов» не имела единого цифрового канала взаимодействия с клиентами, все процессы оформления заказов выполнялись вручную, что приводило к значительным трудозатратам персонала и ошибкам. Определены цель и задачи проекта, объект и предмет работы, используемые технологии, практическая значимость полученных результатов. Приведена краткая характеристика структуры дипломного проекта.

1. Аналитической части. Целью аналитической части является рассмотрение существующего состояния предметной области на примере ООО «Семь драконов» — оптовой компании, специализирующейся на поставке детских игрушек и сопутствующих товаров. Дана характеристика экономического объекта и аппарата управления, рассмотрены бизнес-процессы взаимодействия с клиентами, выявлены проблемы и недостатки в существующей системе управления заказами, обоснованы предложения по их устранению путём разработки цифровой платформы оптовых закупок.

Содержание аналитической части:

- 1.1 Технико-экономическая характеристика предметной области
 - 1.1.1 Характеристика предприятия и подразделения
 - 1.1.2 Экономическая сущность разрабатываемой системы
- 1.2 Анализ существующих решений и аналогов
 - 1.2.1 Сравнительный анализ ключевых аналогов
 - 1.2.2 Выводы по недостаткам существующих решений
- 1.3 Обоснование необходимости разработки
- 1.4 Постановка задач разработки
 - 1.4.1 Цель и назначение системы
 - 1.4.2 Функциональные и нефункциональные требования
- 1.5 Обоснование проектных решений
 - 1.5.1 Техническое обеспечение
 - 1.5.2 Программное обеспечение
 - 1.5.3 Информационное обеспечение
 - 1.5.4 Технологическое обеспечение

2. Проектная часть. Целью проектной части является описание решений, принятых по разработке цифровой платформы. Обоснован выбор стека технологий (React.js, Node.js/Express, PostgreSQL, Drizzle ORM, JWT, Replit), спроектирована клиент-серверная архитектура системы, разработана база данных, смоделированы бизнес-процессы, описаны функциональные модули: онлайн-каталог, личный кабинет, система заказов, чат поддержки, административная панель.

Содержание проектной части:

- 2.1 Информационное и функциональное моделирование
 - 2.1.1 Диаграмма вариантов использования
 - 2.1.2 Диаграмма потоков данных
 - 2.1.3 Диаграмма взаимодействия Frontend и Backend
- 2.2 Проектирование данных
 - 2.2.1 Информационная модель
 - 2.2.2 Характеристика базы данных
 - 2.2.3 Описание нормативно-справочной, входной и результатной информации
- 2.3 Архитектура системы
 - 2.3.1 Диаграмма компонентов
 - 2.3.2 Диаграмма развертывания
- 2.4 Разработка программных модулей
 - 2.4.1 Клиентский модуль
 - 2.4.2 Серверный модуль
 - 2.4.3 Модуль работы с данными
- 2.5 Технологическое обеспечение разработки
 - 2.5.1 Процесс тестирования
 - 2.5.2 Процесс развертывания
 - 2.5.3 Мониторинг и логирование

3. Руководства пользователя В данном разделе описан порядок работы с системой для трёх категорий пользователей: клиента, менеджера и администратора. Приведены системные требования, порядок установки и настройки, описаны все функциональные возможности. Представлены результаты тестирования с описанием выявленных и устранённых ошибок.

Содержание раздела:

- 3.1 Руководство по установке и настройке
 - 3.1.1 Системные требования
 - 3.1.2 Пошаговая инструкция развертывания
 - 3.1.3 Настройка окружения и интеграций
- 3.2 Описание контрольного примера
 - 3.2.1 Сценарий использования системы
 - 3.2.2 Демонстрация работы ключевых функций
- 3.3 Результаты тестирования
 - 3.3.1 Покрытие и виды тестов
 - 3.3.2 Выявленные и устраненные ошибки

4. Экономическое обоснование. Цель раздела — показать эффективность проекта. Рассчитаны затраты на разработку: трудоёмкость и стоимость работ, расходы на ПО и оборудование (ноутбуки, Replit Pro), накладные расходы. Оценён экономический эффект для разработчика (срок окупаемости $\approx 1,3$ года) и для заказчика (годовая экономия 18 628 BYN, срок окупаемости ≈ 15 месяцев). Приведены сводные показатели экономической эффективности.

Содержание экономического обоснования:

- 4.1 Расчет затрат на разработку
 - 4.1.1 Трудоемкость и стоимость работ
 - 4.1.2 Затраты на ПО, оборудование, услуги

4.2 Оценка экономического эффекта

4.2.1 Для разработчика

4.2.2 Для заказчика

4.3 Показатели экономической эффективности

Заключения. В заключении определены задачи, которые были решены в ходе выполнения дипломного проекта, намечены пути внедрения системы на предприятии и направления дальнейшего совершенствования информационной системы.

Перечень графического материала (с точным указанием обязательных чертежей и графиков). Структура организации ООО «Семь драконов»; диаграмма вариантов использования системы; диаграмма потоков данных; диаграмма взаимодействия Frontend и Backend; информационная модель (ER-диаграмма) базы данных; диаграмма компонентов системы; диаграмма развертывания; сравнительный анализ аналогов; демонстрация ключевых функций системы (скриншоты); результаты тестирования; диаграмма распределения трудозатрат по этапам разработки; сравнительная диаграмма трудозатрат до и после внедрения системы.

Консультанты по дипломному проекту с указанием относящихся к ним разделов: аналитический раздел — сотрудники предприятия ООО «Семь драконов», Серебро И.С.; проектный раздел — Серебро И.С.

Примерный календарный график выполнения дипломного проекта:

Наименование раздела, подраздела	Объём работы, %	Дата выполнения	Подпись руководителя
Глава 1 (Аналитическая часть)	25	19.03.2026	
Глава 2 (Проектная часть)	25	08.04.2026	
Глава 3 (Реализация, тестирование и	20	21.04.2026	
Глава 4 (Экономическое обоснование)	20	27.04.2026	
Введение, заключение, список источников, приложения	10	03.05.2026	

1. Дата выдачи задания «20» марта 2026 г.
2. Срок сдачи законченного дипломного проекта «15» мая 2026 г.

Руководитель _____ / _____ /

Обучающийся _____ / _____ /

Дата «20» марта 2026 г

РЕФЕРАТ

Пояснительная записка дипломного проекта содержит 81 страниц, 4 таблиц, 26 формул, 21 иллюстраций, 30 источников литературы.

ЦИФРОВАЯ ПЛАТФОРМА ОПТОВЫХ ЗАКУПОК, ВЕБ-ПРИЛОЖЕНИЕ, REACT.JS, NODE.JS, EXPRESS, POSTGRESQL, DRIZZLE ORM, REST API, JWT, REPLIT, АВТОМАТИЗАЦИЯ В2В-ПРОЦЕССОВ, ООО «СЕМЬ ДРАКОНОВ».

Объект дипломного проекта – цифровая платформа оптовых закупок товаров ООО «Семь драконов» и процессы взаимодействия предприятия с клиентами через цифровые каналы.

Предмет дипломного проекта – методы и инструменты проектирования и разработки корпоративных веб-платформ для автоматизации В2В-процессов оптовой торговли.

Цель дипломного проекта – разработка цифровой платформы оптовых закупок товаров для ООО «Семь драконов» с внедрением онлайн-каталога продукции, личного кабинета клиента, системы онлайн-заказов, чата поддержки и аналитической отчётности.

Методология проведения работы – при выполнении дипломного проекта применялись методы системного анализа деятельности предприятия, функционального моделирования бизнес-процессов, объектно-ориентированного проектирования с использованием ER-диаграмм и UML-диаграмм, сравнительного анализа существующих программных решений, а также методы технико-экономического обоснования проектных решений.

Результат работы – рассмотрена деятельность ООО «Семь драконов», осуществляющей оптовую торговлю детскими игрушками и сопутствующими товарами; выявлены недостатки существующих процессов: отсутствие онлайн-каталога с фильтрацией, личного кабинета клиента, системы онлайн-заказов. Проведён анализ конкурентных решений, обоснована необходимость разработки. Спроектирована клиент-серверная архитектура системы на стеке React.js + Node.js/Express + PostgreSQL; разработана база данных, включающая взаимосвязанные таблицы пользователей, товаров, заказов, чатов и сообщений. Реализованы функциональные модули: онлайн-каталог с поиском и фильтрацией, личный кабинет клиента, оформление и отслеживание заказов, чат поддержки, административная панель управления. Проведён расчёт экономической эффективности: общие затраты на разработку составили 23 927 BYN, годовая экономия для заказчика – 18 628 BYN, срок окупаемости – около 1, 3 года.

Область применения результатов – разработанная цифровая платформа предназначена для использования в ООО «Семь драконов» и обеспечивает автоматизацию взаимодействия предприятия с клиентами из В2В-сегмента: оптовый каталог с ценовыми уровнями, оформление и сопровождение заказов, чат с менеджером, аналитика по продажам. Архитектура системы допускает

дальнейшее масштабирование и подключение новых модулей без переработки серверной части.

Студент-дипломник подтверждает, что приведённый в дипломном проекте расчётно-аналитический материал объективно отражает состояние разрабатываемого объекта, все заимствованные из литературных источников теоретические и методологические положения и концепции сопровождаются ссылками на их авторов.

(подпись студента)

ОГЛАВЛЕНИЕ

СПИСОК ИСПОЛЬЗУЕМЫХ АББРЕВИАТУР И СОКРАЩЕНИЙ.....	9
ВВЕДЕНИЕ.....	10
ГЛАВА 1. АНАЛИТИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ.....	12
1.1 Техничко-экономическая характеристика предметной области.....	12
1.1.1 Характеристика предприятия и подразделения.....	12

1.1.2 Экономическая сущность разрабатываемой системы.....	13
1.2 Анализ существующих решений и аналогов.....	14
1.2.1 Сравнительный анализ ключевых аналогов.....	14
1.2.2 Выводы по недостаткам существующих решений.....	15
1.3 Обоснование необходимости разработки.....	16
1.4 Постановка задач разработки.....	17
1.4.1 Цель и назначение системы.....	17
1.4.2 Функциональные и нефункциональные требования.....	18
1.5 Обоснование проектных решений.....	19
1.5.1 Техническое обеспечение.....	19
1.5.2 Программное обеспечение.....	21
1.5.3 Информационное обеспечение.....	23
1.5.4 Технологическое обеспечение.....	25
ГЛАВА 2. ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА СИСТЕМЫ.....	29
2.1 Информационное и функциональное моделирование.....	29
2.1.1 Диаграмма вариантов использования.....	29
2.1.2 Диаграмма потоков данных.....	30
2.1.3 Диаграмма взаимодействия Frontend и Backend.....	31
2.2 Проектирование данных.....	35
2.2.1 Информационная модель.....	35
2.2.2 Характеристика базы данных.....	37
2.2.3 Описание нормативно-справочной, входной и результатной информации.....	39
2.3 Архитектура системы.....	40
2.3.1 Диаграмма компонентов.....	40
2.3.2 Диаграмма развертывания.....	41
2.4 Разработка программных модулей.....	43
2.4.1 Клиентский модуль.....	43
2.4.2 Серверный модуль.....	44
2.4.3 Модуль работы с данными.....	46
2.5 Технологическое обеспечение разработки.....	47
2.5.1 Процесс тестирования.....	47
2.5.2 Процесс развертывания.....	47
2.5.3 Мониторинг и логирование.....	48
ГЛАВА 3. РЕАЛИЗАЦИЯ, ТЕСТИРОВАНИЕ И ЭКСПЛУАТАЦИЯ.....	50
3.1 Руководство по установке и настройке.....	50
3.1.1 Системные требования.....	50

3.1.2 Пошаговая инструкция развертывания.....	51
3.1.3 Настройка окружения и интеграций.....	51
3.2 Описание контрольного примера.....	53
3.2.1 Сценарий использования системы.....	53
3.2.2 Демонстрация работы ключевых функций.....	55
3.3 Результаты тестирования.....	62
3.3.1 Покрытие и виды тестов.....	62
3.3.2 Выявленные и устраненные ошибки.....	66
ГЛАВА 4. ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ.....	69
4.1 Расчет затрат на разработку.....	69
4.1.1 Трудоемкость и стоимость работ.....	69
4.1.2 Затраты на ПО, оборудование, услуги.....	70
4.2 Оценка экономического эффекта.....	71
4.2.1 Для разработчика.....	71
4.2.2 Для заказчика	73
4.3 Показатели экономической эффективности.....	74
ЗАКЛЮЧЕНИЕ.....	76
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	78
ПРИЛОЖЕНИЕ А.....	80

СПИСОК ИСПОЛЬЗУЕМЫХ АББРЕВИАТУР И СОКРАЩЕНИЙ

ПО	–	программное обеспечение
СУБД	–	система управления базами данных
ТО	–	техническое обеспечение
ТехнО	–	технологическое обеспечение
ИО	–	информационное обеспечение
НДС	–	налог на добавленную стоимость
PL/SQL	–	процедурное расширение языка структурированных запросов (от англ. Procedural Language/Structured Query Language)
TCP	–	протокол управления передачей (от англ. Transmission Control Protocol)
ПС	–	программное средство
ПП	–	программный продукт
PostgreSQL	–	открытая реляционная система управления базами данных

ВВЕДЕНИЕ

Актуальность темы дипломного проекта обусловлена необходимостью цифровизации процессов оптовой торговли и повышения качества взаимодействия ООО «Семь драконов» с корпоративными клиентами. Предприятие осуществляет оптовые поставки детских игрушек и сопутствующих товаров, а значительная часть операций по подбору товарных позиций, согласованию условий поставки, проверке остатков и оформлению заказов выполняется с участием менеджеров вручную. Такой подход увеличивает трудозатраты персонала, повышает вероятность ошибок при расчёте стоимости и замедляет обработку заявок.

В условиях роста конкуренции на рынке детских товаров для оптового предприятия важно обеспечить клиентам удобный цифровой канал закупок: онлайн-каталог с актуальными остатками и ценовыми уровнями, личный кабинет клиента, оформление и отслеживание заказов, оперативную поддержку и административные инструменты управления ассортиментом. Разработка собственной платформы позволяет учесть специфику B2B-продаж ООО «Семь драконов», включая индивидуальные условия для партнёров, объёмные скидки, контроль статусов заказа и подготовку аналитической информации для руководства.

Цель дипломного проекта – разработать цифровую платформу оптовых закупок товаров для организаций на примере ООО «Семь драконов», обеспечивающую автоматизацию клиентского взаимодействия, оформления заказов, управления каталогом, поддержки пользователей и анализа результатов продаж.

Для достижения поставленной цели необходимо решить следующие задачи:

- изучить деятельность ООО «Семь драконов» и выявить недостатки существующего порядка обработки оптовых заказов;
- провести анализ существующих цифровых решений и определить требования к разрабатываемой платформе;
- обосновать проектные решения по техническому, программному, информационному и технологическому обеспечению;
- спроектировать архитектуру системы, структуру базы данных и основные функциональные модули;
- реализовать клиентскую и серверную части веб-приложения с использованием React.js, Node.js/Express, PostgreSQL и Drizzle ORM;
- подготовить руководство по установке, настройке и эксплуатации системы;

– выполнить тестирование разработанной платформы и описать результаты контрольного примера;

– рассчитать экономическую эффективность внедрения системы для разработчика и заказчика.

Объектом дипломного проекта являются процессы взаимодействия ООО «Семь драконов» с корпоративными клиентами при оформлении оптовых закупок товаров.

Предметом дипломного проекта являются методы, модели и программные средства проектирования и разработки веб-платформы для автоматизации B2B-процессов оптовой торговли.

Практическая значимость проекта заключается в создании программного решения, которое может быть использовано ООО «Семь драконов» для снижения трудоёмкости обработки заказов, повышения прозрачности клиентского взаимодействия, уменьшения количества ошибок и формирования единого информационного пространства для менеджеров, клиентов и администратора системы.

Дипломный проект состоит из введения, четырёх глав, заключения, списка использованных источников и приложения. 36 В первой главе приведён анализ предметной области и обоснована необходимость разработки. Во второй главе описаны проектные решения, архитектура, модель данных и программные модули. В третьей главе представлено руководство по установке, контрольный пример и результаты тестирования. В четвёртой главе выполнено экономическое обоснование эффективности проекта.

ГЛАВА 1

АНАЛИТИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ

1.1 Технико-экономическая характеристика предметной области

1.1.1 Характеристика предприятия и подразделения

Общество с ограниченной ответственностью «Семь драконов» – белорусская компания, работающая на рынке оптовой торговли детскими игрушками и сопутствующими товарами. Организация успешно функционирует в Минске, обеспечивая поставки широкого ассортимента продукции для магазинов, торговых сетей и розничных точек по всей стране. С момента основания предприятие делает ставку на надёжные каналы поставок, проверенные бренды и выстроенные деловые связи с партнёрами, что позволило занять устойчивую позицию в сегменте оптовой дистрибуции.

Организационная структура «Семи драконов» построена таким образом, чтобы охватывать весь цикл взаимодействия с клиентами – от приёма заказа и контроля складских остатков до организации логистики и сопровождения сделки. На рисунке 1.1 представлена схема подразделений, обеспечивающих работу ключевых бизнес-процессов.

Основные направления деятельности

- Оптовые поставки игрушек и детских товаров от отечественных и зарубежных производителей.
- Формирование и поддержание широкого ассортимента: классические игрушки, развивающие игры, настольные комплекты, товары для творчества.
- Комплексное складское обслуживание: хранение, учёт, комплектация партий, подготовка заказов к отгрузке.
- Организация доставки по Минску и регионам, включая взаимодействие с транспортными компаниями и контроль сроков.
- Информационная и консультационная поддержка партнёров, помощь в подборе товарных позиций и формировании заказов.

Клиентами компании являются розничные магазины, интернет-ритейлеры, торговые сети и индивидуальные предприниматели. «Семь драконов» придерживается политики гибкого ценообразования и готовности формировать индивидуальные коммерческие предложения для долгосрочных партнёров. Это позволяет обеспечивать конкурентоспособность и сохранять лояльность заказчиков.

Конкурентные преимущества

- Отлаженные бизнес-процессы, минимизирующие сроки обработки заказов и отгрузки.
- Современная складская инфраструктура с возможностью внедрения ИО и автоматизированных систем учёта.
- Ассортимент, адаптированный под потребности различных сегментов рынка.
- Надёжные деловые связи с производителями, что гарантирует качество поставляемой продукции.

Структура ООО «Семь драконов» представлена на рисунке 1.1.



Рисунок 1.1 – Структура организации ООО «Семь драконов»

Примечание – Источник – Собственная разработка.

1.1.2 Экономическая сущность разрабатываемой системы

Разрабатываемая веб-платформа представляет собой цифровой инструмент автоматизации процессов оформления заказов, управления ассортиментом и логистики в ООО «Семь драконов». Её цель – снизить операционные затраты и повысить доходность компании за счёт ускорения внутренней обработки заявок и прозрачного взаимодействия с клиентами.

По своему экономическому эффекту платформа схожа с онлайн-маркетплейсом, объединяющим поставщиков и покупателей, или с конфигуратором подарочных наборов, позволяющим клиентам самостоятельно собирать комплекты игрушек и мгновенно получать актуальную стоимость. В интерфейс инкапсулированы все бизнес-правила по ценообразованию, учёту складских остатков и выбору опций, что обеспечивает быстроту и точность формирования коммерческих предложений.

Экономический эффект достигается за счёт:

- сокращения трудозатрат на ввод заказов и ручную обработку сопроводительной документации
- уменьшения числа ошибок при расчёте цен и проверке наличия товара
- ускорения цикла «заказ – сборка – отгрузка», что положительно влияет на движение денежных средств
- роста клиентской лояльности и увеличения объёма кросс-продаж благодаря персонализированным рекомендациям
- получения централизованной аналитики для оптимизации маркетинговых и складских расходов

В результате внедрения платформы ожидается увеличение выручки за счёт более высокой конверсии заказов, снижение операционных издержек и улучшение ключевых финансовых показателей ООО «Семь драконов» как в краткосрочной, так и в долгосрочной перспективе.

1.2 Анализ существующих решений и аналогов

1.2.1 Сравнительный анализ ключевых аналогов

Таблица 1.1 – Сравнительный анализ ключевых аналогов

Сеть	Формат	Каналы продаж	Цифровые функции	Лояльность и USP	Примечание для «Семь драконов»
Buslik	Оффлайн+онлайн	Сайт, мобильное приложение, 80+ магазинов	Click&Collect, курьерская доставка, личный кабинет, push-уведомления	Бонусная программа, персональные подборки	Пример мультимодальной логистики и WMS-интеграции
KariKids	Оффлайн+онлайн	Сайт, 5 магазинов в Минске	Онлайн-заказ с учётом примерки, отображение остатков	Накопительные скидки, стилистические консультации	Модель локального ритейлера с CRM-коннекторами
FUNtastik	Оффлайн+онлайн	Сайт, офлайн-бутики	Электронный каталог, самовывоз, доставка курьером	Промо-коды, сезонные подборки	Аналог управления ассортиментом и логистикой
Детский мир	Онлайн	Маркетплейс, собственный веб-магазин	Конфигуратор подарков, мгновенный расчёт стоимости	Эксклюзивные брендовые коллекции, быстрая доставка по Беларуси	Демонстрация сложной прайс-модели и SLA-контроля
Mothercare	Оффлайн+онлайн	Сайт, флагманский магазин в Минске	Онлайн-заказ, pick-up point	Бонусные рубли, мероприятия и семинары для родителей	Полезный кейс по управлению сезонными остатками
Платформа ООО «Семь драконов»	B2B / оптовая торговля	Веб-портал, API-интеграция с ERP, WMS, CRM	Автооформление заказов, конфигуратор наборов, SLA-мониторинг	Персонализированные B2B-коммерческие предложения, контроль SLA	Фокус на wholesale-логистике игрушек и отчетности

Примечание: Источник – Собственная разработка.

При сравнении ключевых цифровых решений детских магазинов Беларуси раскрываются следующие преимущества у каждого игрока рынка.

Buslik демонстрирует зрелую омниканальную модель: более 80 офлайн-точек дополняются сайтом и мобильным приложением с поддержкой Click&Collect, курьерской доставки и персонального кабинета. Это позволяет клиентам выбирать удобную схему получения товара, а оператору – гибко управлять запасами через интеграцию с WMS и мобильными сканерами на складе.

Kari Kids делает ставку на глубоко проработанный пользовательский опыт при примерке. В их онлайн-системе клиенты могут заранее увидеть доступность размеров в каждом магазине, выбрать услугу «примерка перед оплатой» и оформить заказ с учётом этого функционала. Такой подход снижает возвраты и повышает конверсию в офлайн-точках.

FUNtastik выигрывает за счёт обширного электронного каталога, где товары структурированы по темам, наборам и подарочным акциям. Инструмент «самовывоз + курьер» на сайте дополняется удобным фильтром аксессуаров и опций, а система промо-кодов и сезонных подборок стимулирует повторные покупки.

Детский мир выделяется конфигуратором подарков – клиенты могут собирать наборы из разных категорий с мгновенным пересчетом стоимости и доступности по регионам. Ускоренная доставка по Беларуси и SLA-контроль доставки «до двери» создают ощущение высокой надёжности сервиса.

Mothercare сочетает офлайн-экспертизу с программой лояльности: бонусные рубли за покупки, тематические события и консультации для родителей. Пик-ап-поинты и онлайн-заказ дополняют формат флагманского магазина, а управление сезонными остатками позволяет снижать издержки на уценку.

1.2.2 Выводы по недостаткам существующих решений

Анализ ключевых цифровых платформ детских магазинов показал, что ни одно из существующих решений не закрывает полностью потребности оптового бизнеса «Семь драконов». Большинство сервисов акцентированы на B2C-сценариях или упрощённых B2B-механизмах, однако для крупного дистрибьютора игрушек требуются более глубокие возможности кастомизации и интеграции.

Основные недостатки конкурирующих решений:

- Ограниченная B2B-поддержка Большинство платформ не умеют формировать индивидуальные коммерческие предложения, не предлагают гибкий учёт объёмных заказов и отсутствуют механизмы автоматического расчёта скидок по уровням партнёров.

– Отсутствие сквозного SLA-мониторинга Решения не дают возможности отслеживать этапы «заказ – сборка – отгрузка» в реальном времени и не уведомляют о нарушении целевых сроков на каждом этапе.

– Недостаточная интеграция с WMS/ERP/CRM Зачастую функционал складывается из базовых API или сторонних модулей, что приводит к точечным доработкам с длительным сроком реализации.

– Слабая аналитическая отчётность Аналитические инструменты ограничены отчётами по продажам и клиентам, не покрывают аналитику остатков, рентабельности по сегментам и прогнозирование спроса.

– Низкая гибкость ценового конфигуратора У большинства аналогов нет удобных инструментов для быстрого формирования комплектов товаров с учётом остатка на складе и оптовых условий.

– Высокие затраты на внедрение и поддержку Решения с развитым BPM-движком или богатой API требуют значительных инвестиций в лицензии и участие сертифицированных консультантов.

– Ограниченный мобильный и офлайн-функционал Отсутствие мобильных приложений для менеджеров и нерешённые задачи офлайн-продаж усложняют оперативную обработку заказов на выезде и врозницу.

По итогу, для «Семь драконов» критически важно разработать платформу с полной поддержкой, сквозным мониторингом, модульной интеграцией с ключевыми системами, гибким конфигуратором наборов и встроенной аналитикой.

1.3 Обоснование необходимости разработки

В современных условиях оптовой торговли игрушками ООО «Семь драконов» сталкивается с расширяющимся портфелем клиентов, усложнённым ассортиментом и возросшими требованиями к скорости обслуживания. Типовые решения, ориентированные на B2C или упрощённое B2B, не позволяют обеспечить необходимый уровень гибкости, прозрачности и контроля, что приводит к:

- высоким трудозатратам на ручную обработку заказов и документов;
- рискам ошибок при расчёте цен и проверке остатков;
- замедлению цикла «заказ – сборка – отгрузка»;
- недостаточной скорости реакции на нестандартные запросы крупных партнёров.

Одновременно усиливается конкурентное давление со стороны как локальных, так и международных дистрибьюторов, внедряющих собственные цифровые платформы. Для сохранения и укрепления позиций на рынке необходимо:

– Автоматизировать полноценный B2B-цикл с возможностью формирования индивидуальных коммерческих предложений и гибким SLA-мониторингом.

– Интегрировать веб-портал с существующими ERP, WMS и CRM-системами без точечных доработок, обеспечив единое информационное пространство.

– Внедрить конфигуратор товарных комплектов, учитывающий остатки на складах и оптовые скидки, для мгновенного расчёта и оформления заказов.

– Обеспечить централизованную BI-аналитику по продажам, рентабельности и прогнозированию спроса, чтобы оперативно корректировать маркетинговые и операционные стратегии.

Разработка собственной платформы позволит ООО «Семь драконов» отказаться от компромиссных решений, устранить узкие места в процессах, минимизировать зависимость от сторонних вендоров и гибко масштабировать функциональность в соответствии с растущими потребностями бизнеса.

1.4 Постановка задач разработки

1.4.1 Цель и назначение системы

Система призвана стать центральным цифровым ядром, объединяющим все ключевые процессы оптовой торговли ООО «Семь драконов». Она обеспечит единый интерфейс для оформления заказов, управления ассортиментом и контроля логистических операций, что снимет необходимость переключаться между разрозненными приложениями и значительно упростит работу как внутренних пользователей, так и клиентов компании. В основе платформы лежит гибкая архитектура, способная адаптироваться под растущие потребности бизнеса и новые сценарии взаимодействия с партнёрами.

Одной из главных функциональных задач станет автоматизированный конфигуратор товарных комплектов. Клиенты получают возможность собирать заказы из различных категорий продукции, сразу видя актуальные остатки на складах, стоимость с учётом оптовых скидок и варианты доставки. Такой подход не только ускорит процесс формирования коммерческих предложений, но и снизит количество ошибок, присущих ручному подбору и многоступенчатым согласованиям.

Полный цикл «заказ – сборка – отгрузка» будет полностью прозрачен: система будет фиксировать каждый этап выполнения заявки, автоматически рассчитывать SLA-показатели и оповещать ответственных лиц о возможных нарушениях сроков. Это позволит менеджерам оперативно вмешиваться в процесс при возникновении задержек и обеспечит соблюдение обязательств

перед ключевыми партнёрами компании. Переход от ручного контроля к автоматизированному мониторингу статусов сделает работу более предсказуемой и сократит риски возникновения «узких мест».

Реализация данной платформы обеспечит ощутимое снижение операционных расходов за счёт минимизации ручного ввода и рутинных согласований, повысит скорость денежного цикла за счёт ускоренного обслуживания заказов и укрепит лояльность ключевых клиентов благодаря соблюдению прозрачным условиям работы. При этом масштабируемая архитектура и поддержка современных веб-технологий создадут фундамент для дальнейшего развития системы без значительных дополнительных затрат на IT-ресурсы.

1.4.2 Функциональные и нефункциональные требования

Функциональные и нефункциональные требования к будущей платформе для ООО «Семь драконов» можно развернуть в более широкое, комплексное описание, чтобы подчеркнуть масштаб проекта и важность каждого элемента.

Платформа должна стать единым цифровым центром, охватывающим все этапы взаимодействия с клиентами и внутренними подразделениями. Её функциональность начинается с гибкого управления пользователями: система будет поддерживать регистрацию, аутентификацию, назначение ролей и настройку прав доступа таким образом, чтобы каждая категория пользователей – от менеджера по продажам до кладовщика и клиента – имела чётко определённый набор инструментов. Это позволит исключить избыточный доступ, усилить безопасность и ускорить работу за счёт персонализированных интерфейсов.

Процесс оформления и обработки заказов станет полностью автоматизированным. Менеджер сможет создать заказ вручную или через интуитивный конфигуратор товарных наборов, который в реальном времени покажет остатки, рассчитает стоимость с учётом скидок, валютных курсов и акций, а также предложит альтернативы при дефиците товаров. Многоуровневая система согласований обеспечит контроль ценовых условий, доступности и сроков поставки, а история заказов позволит быстро анализировать прошлые сделки и повторять успешные конфигурации.

Отдельный модуль будет отвечать за управление прайс-листами и скидками, включая сезонные акции, персональные условия для ключевых клиентов и автоматическое пересчитывание цен при изменении условий поставок или рыночных факторов. SLA-мониторинг станет важной частью платформы: он не только проконтролирует соблюдение сроков на каждом этапе

цикла «заказ – сборка – отгрузка», но и вовремя предупредит о рисках срыва обязательств через встроенную систему уведомлений.

С точки зрения интеграций, система будет «общаться» с ERP для обмена данными о номенклатуре и остатках, с WMS – для передачи заданий на сборку и контроля перемещений на складе, и с CRM – для управления клиентской базой и отслеживания истории взаимодействий. Все эти интеграции будут реализованы через стандартизированные API, что упростит подключение новых модулей и снизит стоимость поддержки. BI-подсистема соберёт данные из всех точек и предоставит руководству аналитические отчёты и прогнозы, позволяя мгновенно оценивать эффективность продаж и корректировать стратегию.

В нефункциональной части платформа должна быть высокой производительности, способной обрабатывать тысячи заказов в сутки без потери скорости отклика. Уровень доступности планируется не ниже 99,9 %, при этом будет реализовано горячее резервирование и регулярное бэкапирование данных. Безопасность обеспечат шифрование трафика, защищённое хранение конфиденциальных данных и регулярные тесты на уязвимости. Интерфейс будет адаптивным, многоязычным и соответствующим стандартам доступности, что позволит использовать систему на разных устройствах и в разных условиях.

Модульная архитектура и современный технологический стек обеспечат лёгкое масштабирование и развитие платформы по мере роста бизнеса. Таким образом, это решение не просто автоматизирует существующие процессы, но и создаст основу для дальнейших цифровых инициатив «Семь драконов», обеспечивая компании конкурентное преимущество на долгие годы.

1.5 Обоснование проектных решений

1.5.1 Техническое обеспечение

Техническое обеспечение (ТО) платформы учёта и автоматизации для ООО «Семь драконов» – это комплекс аппаратных средств, сетевого оборудования и инженерной инфраструктуры, обеспечивающих стабильную и безопасную работу ПО, СУБД и сервисных модулей системы. Основная задача ТО – гарантировать требуемый уровень производительности, отказоустойчивости и информационной безопасности при работе в режиме 24/7.

Серверная инфраструктура
– Веб-сервер

- CPU: Intel Xeon E5-2680 v4 (14 ядер, 2.4 ГГц) или аналог • ОЗУ: 32 ГБ DDR4
- Накопитель: SSD ≥ 1 ТБ Обоснование: высокая многопоточность и большой объём памяти позволяют обрабатывать сотни параллельных запросов, кэшировать данные и снижать время отклика платформы.
 - Сервер СУБД
 - CPU: Intel Xeon Gold 6248 (20 ядер, 2.5 ГГц) или аналог
 - ОЗУ: 64 ГБ DDR4
 - Накопитель: SSD ≥ 2 ТБ в RAID 1 Обоснование: высокая многопоточность и объём памяти критичны для сложных транзакций, формирования аналитических отчётов и работы с большими наборами данных; RAID 1 обеспечивает отказоустойчивость.
 - Канал связи
 - Выделенная линия ≥ 1 Гбит/с с резервированием Обоснование: стабильный высокоскоростной канал исключает узкие места в интеграциях с WMS/CRM, ускоряет синхронизацию складских операций и обеспечивает пиковую нагрузку без деградации.
 - Система бесперебойного питания (ИБП)
 - Мощность ≥ 10 кВА Обоснование: предотвращает аварийную остановку систем и потерю данных при перебоях электроснабжения.
 - Требования к клиентским устройствам
 - Процессор
 - ОЗУ: Intel Core i3 или AMD Ryzen 3 и выше, ОЗУ ≥ 4 ГБ Обоснование: обеспечивает быструю работу интерфейса WMS-модуля, личного кабинета и мобильных приложений для склада.
 - Дисковое пространство: ≥ 100 МБ свободного места
 - Обоснование: для хранения временных файлов, офлайн-кэша и локальных логов.
 - Сетевое подключение: широкополосный Интернет
 - Обоснование: гарантирует синхронность данных с сервером.
 - Поддерживаемые браузеры
 - Chrome 90+
 - Firefox 88+
 - Safari 14+
 - Edge 90+
 - Сетевое оборудование
 - Коммутаторы уровня L3 с пропускной способностью ≥ 48 Гбит/с
 - Обоснование: агрегация и сегментация трафика, поддержка VLAN для изоляции складской, офисной и гостевой сетей.

- Маршрутизаторы с поддержкой BGP/OSPF
 - Обоснование: динамическая маршрутизация и балансировка каналов при сбоях.
- Межсетевые экраны с производительностью ≥ 10 Гбит/с
 - Обоснование: глубокая инспекция пакетов, защита от DDoS и атак на периметр.
- Точки доступа Wi-Fi стандарта 802.11ac/ax
 - Обоснование: высокоскоростной доступ на складе и в офисе, поддержка большого количества одновременных подключений.

1.5.2 Программное обеспечение

Программное обеспечение (ПО) разрабатываемой системы представляет собой взаимосвязанную совокупность системных, прикладных, интеграционных и сервисных компонентов, ориентированных на автоматизацию ключевых бизнес-процессов оптовой торговли и логистики. Оно реализует предметную область проекта, поддерживает внутренние и внешние интеграции, а также обеспечивает удобный, безопасный и производительный доступ пользователей к функционалу системы.

Серверный сегмент формирует ядро бизнес-логики и отвечает за обработку клиентских запросов, интеграцию с внешними модулями и управление данными. Архитектура выстроена на основе асинхронной среды выполнения, что позволяет обслуживать сотни и тысячи одновременных соединений с минимальными задержками. Это особенно важно для задач реального времени: синхронизации складских операций, обработки заказов, обновления аналитики.

Ключевые особенности серверной части:

- Статическая типизация обеспечивает строгий контроль структуры данных на всех этапах разработки, снижает риск логических ошибок и упрощает масштабирование кода.
- Модульная архитектура позволяет разрабатывать, тестировать и внедрять отдельные функциональные блоки независимо, что ускоряет внедрение новых возможностей.
- Слой маршрутизации и middleware служит для централизованной обработки запросов – валидация данных, аутентификация, ведение логов, сжатие и кэширование ответов.
- ORM-уровень обеспечивает безопасную работу с БД: автоматическую генерацию миграций, контроль типов, оптимизацию SQL-запросов и защиту от инъекций.

- Управление сессиями реализовано с использованием устойчивого к сбоям хранилища, что позволяет поддерживать авторизацию в кластере серверов и сохранять состояние пользователей при балансировке нагрузки.

- Интеграционный слой обрабатывает двусторонний обмен с бухгалтерскими системами, сервисами доставки, маркетплейсами и модулями мониторинга.

Пользовательский интерфейс реализован как одностраничное приложение, что обеспечивает мгновенную навигацию и динамическое обновление данных без полной перезагрузки страниц. Декларативный подход и виртуальный DOM гарантируют высокую отзывчивость даже при больших объёмах отображаемой информации.

Отличительные черты клиентской части:

- Единая система типов между сервером и клиентом исключает несогласованность форматов данных.

- Утилитарная система стилей и библиотека готовых компонентов позволяют быстро собирать сложные интерфейсы, сохраняя при этом единый фирменный стиль.

- Минимальный и оптимизированный роутер обеспечивает плавную работу навигации без нагруженности приложения лишними зависимостями.

- Адаптивный дизайн гарантирует корректное отображение на стационарных ПК, ноутбуках, планшетах и мобильных устройствах.

В качестве хранилища применяется промышленная реляционная СУБД с полной поддержкой ACID, оптимизированной системой индексов, механизмами репликации и восстановления после сбоев.

Функциональные преимущества:

- Поддержка сложных транзакций с высокой степенью параллелизма.

- Гибридные структуры данных – реляционные таблицы и JSON-поля для хранения полуструктурированной информации.

- Горизонтальное масштабирование за счёт шардинга и репликации.

- Механизмы резервного копирования для обеспечения целостности и восстановления данных.

Ключевые функциональные модули клиентской стороны:

- Аутентификация, авторизация.

- Формирование, редактирование и отслеживание заказов.

- Просмотр и анализ показателей продаж, остатков.

- Поддержка клиентов.

- Интерактивное отображение статусов заказов.

Ключевые функциональные модули серверной стороны:

- RESTful API для всех операций управления данными и взаимодействия с внешними системами.

- Ролевое разграничение доступа и централизованная авторизация.

- Система логирования и мониторинга для проактивного выявления проблем и оптимизации производительности.

Обеспечение безопасности:

- Шифрование трафика между всеми компонентами.

- Хранение конфиденциальных данных с использованием алгоритмов симметричного шифрования.

- Валидация и фильтрация данных на всех уровнях (сервер и клиент).

Эксплуатационные и архитектурные аспекты:

- Микросервисный подход облегчает масштабирование отдельных частей системы под нагрузкой.

- Контейнеризация обеспечивает идентичность сред разработки, тестирования и эксплуатации.

- Мониторинг (метрики, логи, алерты) обеспечивает предсказуемость работы и высокую доступность.

1.5.3 Информационное обеспечение

Информационное обеспечение корпоративной платформы ООО «Семь драконов» – это комплексная совокупность данных, методов и регламентов, обеспечивающих поддержку всех бизнес-процессов предприятия в цифровой среде. Оно охватывает как содержательную сторону (состав и структуру данных), так и организационную (правила их формирования, обновления, хранения, передачи и защиты). По сути, это «информационный фундамент» компании, позволяющий автоматизировать ключевые функции и выстроить единое информационное пространство между подразделениями.

- Пользовательские данные

- Состав данных: В систему заносятся и системно обрабатываются сведения о клиентах и партнёрах, прошедших регистрацию. В их числе:

- E-mail (основной идентификатор для входа и коммуникаций)

- Название компании (юридическое и/или коммерческое)

- ФИО директора

- Телефон (рабочий или мобильный)

- Почтовый адрес (для выставления счетов и доставки документов)

- Назначение: Эти данные выполняют роль базы идентификации и аутентификации участников системы, лежат в основе документооборота.

- Защита и разграничение прав: Реализована двухуровневая авторизация:

- Пользователи – имеют доступ к личному кабинету, заказам, коммуникационному модулю.

- Администраторы – управляют ассортиментом, заказами, учетными записями, настройками и интеграциями.

- Товарная номенклатура

- Состав данных: Каталог продукции содержит полный набор характеристик по каждой позиции:

- Наименование товара

- Подробное описание

- Категория

- Возрастная группа

- Материал

- Страна производства

- Изображение (фотография)

- Трёхуровневая модель ценообразования:

- 5+ единиц

- 20+ единиц

- 50+ единиц

- Статус наличия (в наличии, под заказ, отсутствует)

- Назначение: Товарная база служит центральным информационным ресурсом, который используют:

- Клиенты – для выбора и оформления заказов

- Менеджеры – для ведения каталога и расчёта цен

- Склад – для синхронизации остатков

- Отдел маркетинга – для подготовки рекламных материалов

- Особенности: Синхронизация с системой управления складом (WMS) обеспечивает актуальность статусов наличия. При изменении остатков обновление отражается в каталоге в режиме реального времени. Алгоритмы ценообразования позволяют автоматически пересчитывать стоимость в корзине, стимулируя оптовые закупки.

- Заказная информация

- Состав данных по каждому заказу:

- Перечень товаров с указанием количества

- Общая стоимость (с учётом ценового уровня и возможных скидок)

- Адрес доставки

- Статус обработки заказа:

- Новый
- В обработке
- Отправлен
- Доставлен
- Планируемая дата доставки

○ Назначение и обработка: Эти сведения обеспечивают прозрачность всего цикла обработки заказа. На каждом этапе обновляется статус, что позволяет клиенту отслеживать движение своей заявки, а менеджерам – планировать загрузку склада и логистики.

1.5.4 Технологическое обеспечение

Технологическое обеспечение разрабатываемой веб-платформы для ООО «Семь драконов» представляет собой комплекс современных технологий, инструментов и библиотек, которые формируют основу клиентской и серверной части системы, обеспечивают её надёжность, производительность, расширяемость и удобство использования. **38** правильно подобранный технологический стек позволяет создать продукт, отвечающий как текущим, так и перспективным потребностям бизнеса, а также упрощает последующее сопровождение и развитие.

В основе пользовательского интерфейса лежит современная библиотека для построения веб-интерфейсов с декларативным подходом к описанию компонентов. Использование актуальной версии этой библиотеки в сочетании с языком программирования, поддерживающим строгую статическую типизацию, позволяет:

- минимизировать количество ошибок на этапе разработки;
- обеспечить предсказуемость поведения кода;
- упростить совместную работу команды над крупными модулями;
- реализовать повторно используемые, модульные интерфейсные компоненты.

Сборка и запуск фронтенд-приложения выполняются с помощью высокопроизводительного инструмента разработки, обеспечивающего мгновенный запуск проекта (cold start), поддержку «горячей» перезагрузки модулей (HMR) и минимальное время отклика во время изменения кода. Это ускоряет цикл разработки, тестирования и внедрения изменений, что особенно важно для динамично развивающегося проекта.

Для управления серверным состоянием и синхронизации данных между клиентом и API используется специализированная библиотека, оптимизирующая запросы, кэширование и повторное использование данных. Это позволяет:

- уменьшить количество избыточных обращений к серверу;
- моментально отображать актуальную информацию при повторных визитах пользователя;
- эффективно обрабатывать ошибки и состояния загрузки.

Маршрутизация внутри одностраничного приложения (SPA) реализована при помощи лёгкого клиентского роутера, что даёт возможность перемещаться между разделами без полной перезагрузки страницы и снижает нагрузку на сервер.

Для унификации внешнего вида и ускорения разработки используется утилитарный CSS-фреймворк, позволяющий создавать адаптивную и согласованную дизайн-систему без написания избыточных каскадных стилей. В связке с ним применяется библиотека готовых компонентов на основе Radix UI, что обеспечивает:

- доступность интерфейса (соответствие стандартам WCAG);
- предсказуемое поведение элементов управления;
- современный внешний вид и поддержку тёмной/светлой тем.

Это решение позволяет не только экономить время разработчиков, но и гарантирует единообразие пользовательского опыта на всех устройствах.

Серверная логика реализована на основе минималистичного веб-фреймворка, работающего в связке с TypeScript. Такой выбор обеспечивает:

- гибкую маршрутизацию HTTP-запросов;
- поддержку middleware-архитектуры для централизованной обработки ошибок, логирования, аутентификации и авторизации;
- простоту интеграции с внешними API и микросервисами.

Для работы с загружаемыми пользователями файлами (изображения товаров, документы и т.д.) используется специализированная библиотека, позволяющая эффективно обрабатывать файлы в памяти или на диске, а также накладывать ограничения на их размер и формат.

Вся бизнес-логика сервера модульно структурирована, что облегчает масштабирование приложения и внедрение новых функций без затрагивания существующих модулей.

В качестве основной системы управления базами данных используется надёжная реляционная СУБД с полной поддержкой транзакций, оптимизированными индексами и возможностью горизонтального масштабирования. Для доступа к БД применяется ORM-слой, который:

- обеспечивает полную типобезопасность запросов;
- автоматически генерирует миграции схемы данных;
- снижает риск SQL-инъекций;
- упрощает рефакторинг структуры данных.

Подключение к базе реализовано через облачный serverless-провайдер с пулом соединений, что позволяет:

- автоматически масштабировать ресурсы под нагрузку;
- оптимизировать время отклика запросов;
- снижать затраты на инфраструктуру за счёт динамического распределения вычислительных мощностей.

Технические и архитектурные преимущества выбранного стека:

– Производительность и масштабируемость – благодаря асинхронной обработке запросов на сервере, оптимизированной работе с БД и эффективному управлению состоянием на клиенте.

– Кроссплатформенность – интерфейс корректно работает в современных браузерах на любых устройствах, включая мобильные.

– Расширяемость – модульная архитектура позволяет легко добавлять новые компоненты и модули.

– Поддержка командной разработки – строгая типизация и единые стандарты кодирования ускоряют взаимодействие между разработчиками.

– Доступность и UX-качество – за счёт использования UI-фреймворка, ориентированного на доступность и адаптивный дизайн.

– Интеграционная готовность – API и архитектура бэкенда позволяют подключать внешние сервисы и корпоративные системы без кардинальной переработки кода.

ГЛАВА 2

ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА СИСТЕМЫ

2.1 Информационное и функциональное моделирование

2.1.1 Диаграмма вариантов использования (Use Case)

Диаграмма вариантов использования – это один из видов поведенческих диаграмм UML, который демонстрирует функциональные требования к системе через “варианты использования” (Use Cases) и внешних участников (Actors). Представлена она на рисунке 2.1

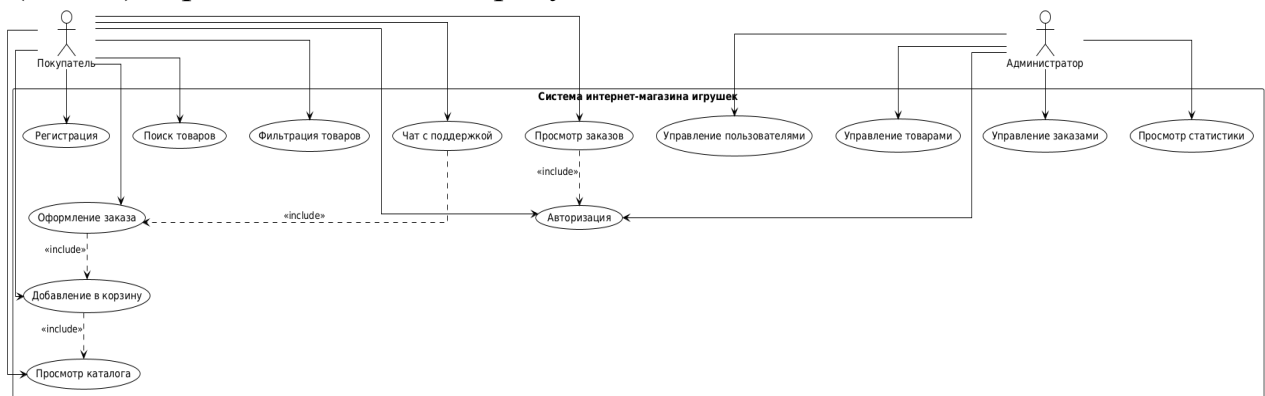


Рисунок 2.1 – Диаграмма вариантов использования.

Примечание – Источник – Собственная разработка.

Покупатель начинает работу с регистрации личного кабинета, после чего получает доступ к поиску и фильтрации товаров, может просматривать каталог и добавлять выбранные позиции в корзину. Для уточнения информации он имеет возможность вести чат с поддержкой, а процесс оформления заказа включает обязательную авторизацию и формирование чата для связи с продавцом. Администратор, напротив, выполняет функции управления: он следит за пользователями, добавляет, редактирует и удаляет учетные записи, управляет каталогом товаров, обновляет описания и наличие, обрабатывает заказы, изменяет их статус и дату доставки, а также анализирует статистику по продажам, заказам и активности клиентов. Таким образом, диаграмма четко показывает разграничение прав и задач между типами пользователей, отражает взаимосвязи между сценариями (например, оформление заказа опирается на авторизацию, а добавление в корзину связано с просмотром каталога) и служит основой для формализации требований при разработке системы.

2.1.2 Диаграмма потоков данных

Диаграмма потоков данных отражает логику прохождения информации и запросов в системе интернет-магазина, показывая, как взаимодействуют основные участники, программные компоненты и хранилища данных.

Представлена она на рисунке 2.2

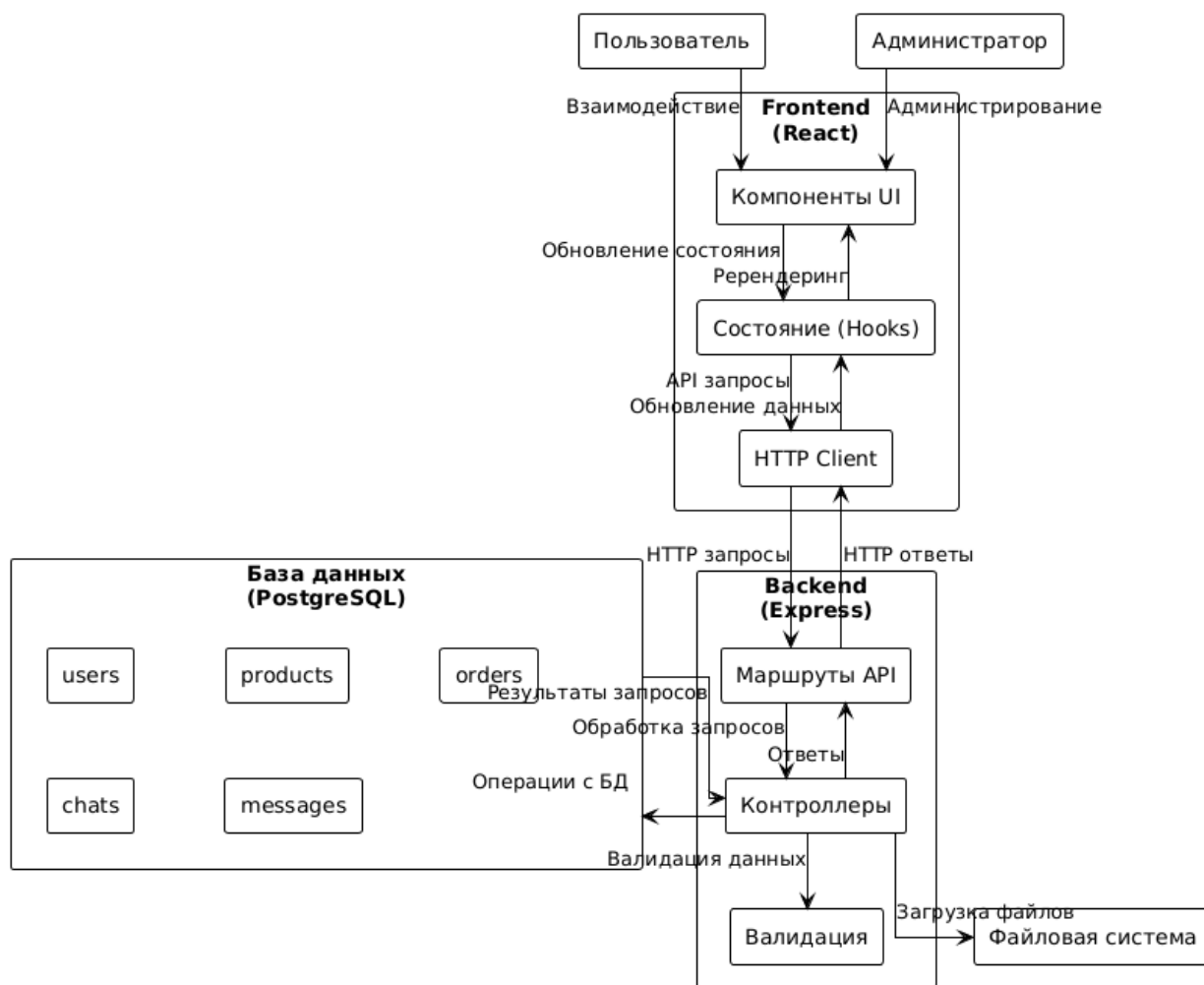


Рисунок 2.2 – Контекстная диаграмма.

Примечание – Источник – Собственная разработка.

Взаимодействие инициируется во фронтенд-приложении на React, где пользовательский интерфейс принимает команды, управляет состоянием и через HTTP-клиент отправляет запросы в формате JSON или FormData на сервер. Серверная часть на Express принимает эти запросы, обрабатывает их в маршрутизаторах и контроллерах, выполняет валидацию и при необходимости обращается к базе данных PostgreSQL или файловой системе. В базе хранятся сведения о пользователях, товарах, заказах, чатах и сообщениях, а в файловой системе – загруженные изображения и другие медиа. В ответ на SQL-операции или команды чтения файлов сервер формирует результаты и возвращает их клиенту для отображения в интерфейсе. Такой цикл характерен для всех сценариев работы: поиск товаров сопровождается GET-запросом и выборкой из

таблицы products, оформление заказа включает авторизацию, вставку записей в orders и создание чата, загрузка изображений администратором сочетает сохранение файла в /uploads с записью метаданных в БД, а обмен сообщениями реализуется вставкой данных в messages и отправкой клиенту. Диаграмма демонстрирует, что система состоит из связанных уровней – клиентского, серверного и уровня хранения – между которыми непрерывно циркулируют структурированные потоки данных, обеспечивая работу и пользовательских, и административных функций.

2.1.3 Диаграмма взаимодействия Frontend и Backend

Диаграмма взаимодействия отражает последовательность обмена запросами и ответами между клиентской частью (React-приложение) и серверной частью системы (Express-сервер с доступом к PostgreSQL и файловой системе), разложенную по ключевым пользовательским сценариям. Представлена она на рисунке 2.3

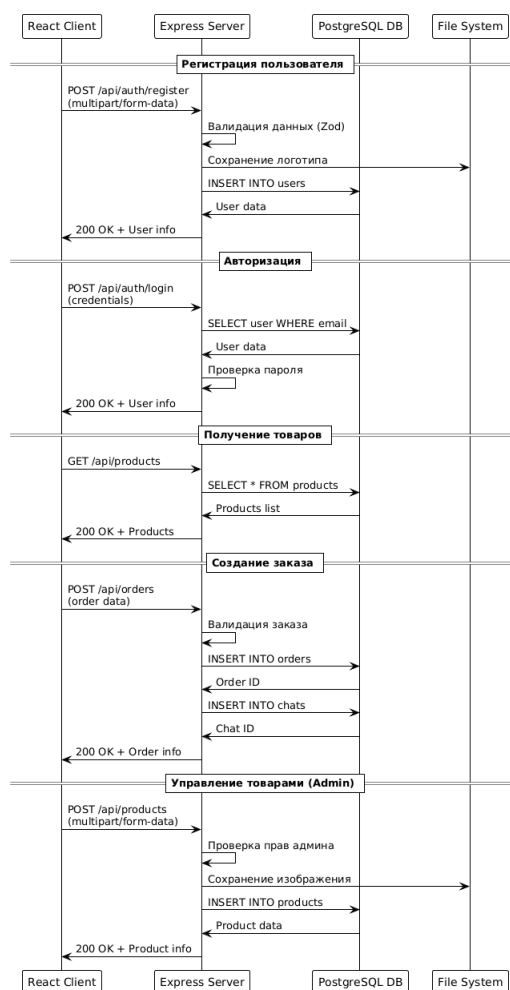


Рисунок 2.3 - Диаграмма взаимодействия Frontend и Backend.

Примечание: Источник – Собственная разработка.

Пользователь или администратор начинает работу с интерфейса, реализованного на React: визуальные компоненты принимают действия, формируют запросы и через HTTP-клиент отправляют их на API-эндпоинты. В сценарии регистрации система принимает от клиента POST-запрос с формой и загружаемым логотипом, на сервере выполняется валидация, файл сохраняется в хранилище, а данные учётной записи – в таблицу пользователей базы PostgreSQL, после чего на клиент возвращается подтверждение.

При авторизации фронтенд отправляет логин и пароль, бэкенд проверяет их по записям в базе, генерирует ответ с учётными данными и правами доступа, и интерфейс обновляет состояние пользователя. Запрос на получение каталога товаров приводит к выборке из таблицы products, результат которой сериализуется в JSON и отображается в интерфейсе. Создание заказа сопровождается отправкой структурированных данных о выбранных товарах, которые бэкенд проверяет, сохраняет в таблице orders, одновременно создаёт связанную запись в chats для общения с продавцом, и возвращает агрегированную информацию об операции. Для административных действий, например добавления товара, клиент передаёт multipart-запрос с изображением и описанием, сервер проверяет права, сохраняет файл в директорию /uploads и вносит соответствующую запись в базу. Во всех случаях диаграмма показывает линейную цепочку шагов – формирование запроса на клиенте, обработку на сервере, обращение к базе или файловой системе, формирование ответа и его возврат в интерфейс. Она также подчёркивает, что между фронтендом и бэкендом идёт постоянный двусторонний обмен: от клиента к серверу передаются данные форм, параметры поиска и команды управления, а обратно возвращаются результаты выборок, статические ресурсы, статусы выполнения и сообщения об ошибках. Эта схема наглядно фиксирует взаимосвязь всех уровней системы и потоков данных, обеспечивающих работу пользовательских и административных функций.

Схема взаимодействия компонентов отражает целостную архитектуру веб-приложения интернет-магазина игрушек, показывая, как все уровни системы – от пользовательского интерфейса до хранилищ данных – соединены в единый рабочий механизм.

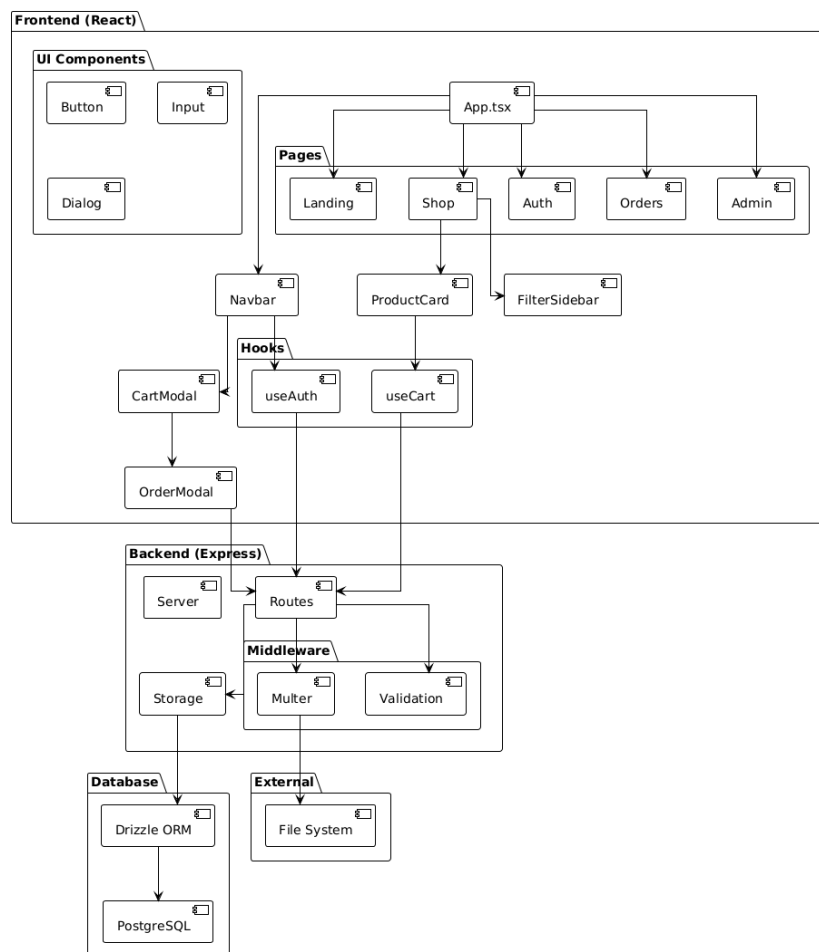


Рисунок 2.4 - Схема взаимодействия компонентов.

Примечание – Источник – Собственная разработка.

На стороне клиента работает фронтенд, реализованный на React, который состоит из набора UI-компонентов вроде кнопок, полей ввода и диалоговых окон, а также более сложных элементов, таких как навигационные панели, карточки товаров и панели фильтров. Эти компоненты объединяются на страницах – от стартовой и каталога до страниц авторизации, заказов и административной панели, – каждая из которых связывает визуальную часть с логикой работы. Состояние приложения управляется через хуки, например useAuth для аутентификации и useCart для корзины, а модальные окна обеспечивают оформление заказа или просмотр содержимого корзины без перехода на другие страницы. Все действия пользователя преобразуются в запросы через HTTP-клиент и отправляются на серверную часть. Бэкенд построен на Express и включает сервер, который через маршрутизацию распределяет входящие HTTP-запросы по контроллерам модулей – аутентификации, товаров, заказов, чатов и администрирования. На этом уровне выполняется бизнес-логика: проверка прав доступа по токену, валидация данных, обработка загрузки файлов с помощью Multer, логирование и обработка ошибок.

Для работы с базой данных используется Drizzle ORM, обеспечивающий удобное сопоставление сущностей на таблицы PostgreSQL, где хранятся пользователи, товары, заказы, чаты и сообщения, с поддержкой транзакций и связей по внешним ключам. Дополнительно в файловой системе, в каталоге /uploads, сохраняются изображения товаров, логотипы и другие медиафайлы, которые отдаются клиенту как статические ресурсы. Потоки данных между фронтендом и бэкендом организованы по REST-принципу: от клиента поступают запросы с параметрами или телом в формате JSON либо multipart/form-data для медиа, а обратно возвращаются ответы с данными или файлами. В сценариях регистрации или входа фронтенд передает данные форм, бэкенд проводит проверку и сохраняет сведения в базе, а затем возвращает токен и профиль; при загрузке каталога отправляется запрос на список товаров, сервер выбирает записи из таблицы и отдает их клиенту; при оформлении заказа создается запись в заказах и чат для общения с продавцом, а при добавлении товара администратором изображение сохраняется в файловой системе, а сведения о товаре – в базе данных.

Вся система организована так, что фронтенд взаимодействует исключительно с API и статическими файлами, бэкенд – только с хранилищами данных, что делает архитектуру модульной, расширяемой и легко сопровождаемой, а схема наглядно отображает, какие компоненты отвечают за каждый участок пути данных и как они связаны между собой в единый поток работы.

В представленной схеме REST API все конечные точки сгруппированы по функциональным областям, что позволяет чётко разграничить задачи между модулями и обеспечить предсказуемость взаимодействия клиента с сервером.

В блоке Authentication API сосредоточены операции, связанные с учётными записями: регистрация нового пользователя посредством POST /api/auth/register с передачей необходимых данных формы, вход в систему через POST /api/auth/login и завершение текущей сессии командой POST /api/auth/logout. 27 методы обеспечивают начальную точку входа и контроля доступа к защищённым ресурсам.

Product API управляет жизненным циклом товаров. Добавление новой позиции выполняется запросом POST /api/products с метаданными и изображением, получение всего каталога – через GET /api/products, а выбор конкретного товара по идентификатору – запросом GET /api/products/{id}. Для актуализации информации предусмотрен PUT /api/products/{id}, а для удаления – DELETE /api/products/{id}.

В Orders API отражены операции работы с заказами: POST /api/orders создаёт новый заказ, PUT /api/orders/{id}/status меняет его статус, GET /api/orders возвращает список всех заказов (с возможностью фильтрации по

ролям и параметрам), а GET /api/orders/{id} **1** даёт полную информацию по конкретному заказу.

Chat API обеспечивает обмен сообщениями между пользователями. Отправка нового сообщения происходит через POST /api/chat/messages, а получение списка переписки – запросом GET /api/chat/messages. Эти действия могут обслуживать как пользовательские, так и административные чаты, в зависимости от бизнес-логики.

В совокупности структура эндпоинтов демонстрирует модульный подход: каждый набор методов инкапсулирует определённую область функционала, соблюдает единообразие форматов и названий, что упрощает интеграцию фронтенда и позволяет быстро расширять API, не нарушая существующие контракты между клиентом и сервером.

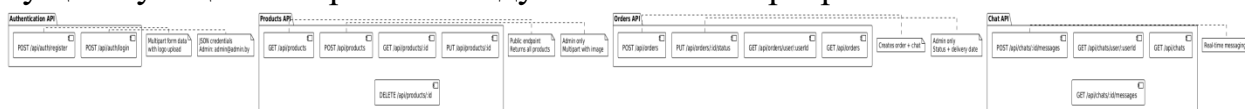


Рисунок 2.5 - REST API Endpoints.

Примечание: Источник – Собственная разработка.

2.2 Проектирование данных

2.2.1 Информационная модель

Информационная модель – это формальное представление ключевых объектов предметной области, их атрибутов и отношений, на основе которого проектируется структура СУБД и обеспечивается целостность данных.

Эта информационная модель описывает целостную структуру базы данных, спроектированную для автоматизации работы компании, которая одновременно управляет каталогом товаров, заказами клиентов и их взаимодействием с поддержкой. В её основе лежит принцип нормализации данных и разделения сущностей по логическим блокам, что позволяет обеспечивать гибкость, масштабируемость и поддерживаемость системы. Центральным элементом модели является сущность User, которая содержит учетные записи всех участников – от обычных клиентов до администраторов. Для каждого пользователя фиксируются как контактные данные и сведения о компании, так и авторизационные параметры и признак административных прав, что позволяет дифференцировать доступ к функциям системы. Уникальность электронной почты и индексация по ключевым признакам повышают скорость поиска и безопасность хранения данных.

Вторым крупным блоком выступает каталог товаров, представленный сущностью Product. Здесь хранятся описания и изображения товаров, их классификация по категориям и возрастным группам, информация о материале

изготовления и стране происхождения. Особое внимание уделено ценовой политике: для разных объемов закупки заданы отдельные поля с прогрессивными скидками. Поля категории и возрастной группы привязаны к справочникам, что исключает дублирование и обеспечивает контроль корректности данных.

Далее идёт заказной сущность Order. Она отражает «шапку» заказа, содержащую сведения о клиенте, общей сумме, адресе доставки, текущем статусе и дате создания. Связанный справочник OrderStatus фиксирует полный жизненный цикл заказа от момента оформления до доставки или отмены. Детализация состава заказа реализована через отдельную таблицу OrderItem, где для каждой позиции указываются товар, количество, цена за единицу и тип ценового уровня. Такой подход упрощает аналитику, позволяет точно рассчитывать себестоимость и дает гибкость при обработке заказов разного масштаба.

Последний функциональный блок посвящен коммуникациям. Каждому заказу автоматически соответствует сущность Chat, связывающая заказ и клиента в рамках одного канала поддержки. Внутри чата хранится множество записей сущности Message, каждая из которых содержит идентификатор чата, отправителя, текст сообщения и дату его отправки. Эта структура обеспечивает прозрачную историю переписки, удобную для обеих сторон. Информационная модель ИС приведена на рисунке 2.8

Характерной особенностью модели является работа как с традиционными реляционными структурами, так и со сложными вложенными данными. Для хранения агрегированных объектов используется тип JSONB – в частности, для первоначального варианта хранения состава заказа. Это позволяет гибко описывать переменное количество элементов, не нарушая общей структуры базы, а также эффективно работать с фильтрацией и выборками по вложенным ключам. Несмотря на то, что в оптимизированной версии модели позиции заказа вынесены в отдельную таблицу OrderItem для аналитической прозрачности, поддержка JSON остаётся ценным инструментом для хранения вспомогательных структур, фильтров и параметров.

Все ключевые сущности системы снабжены временными метками (createdAt, updatedAt), что формирует полноценный аудит-след и позволяет отслеживать историю изменений, жизненный цикл заказов и событий. Данный механизм важен как для внутреннего контроля, так и для аналитики бизнес-процессов.

Логическая модель базы данных охватывает несколько функциональных блоков:

- Пользователи и учетные данные – таблица User с расширенной контактной и организационной информацией, признаком административных прав и уникальной аутентификацией по e-mail.

- Каталог товаров – таблица Product с детальными характеристиками товара, классификацией по категориям и возрастным группам, а также прогрессивной ценовой матрицей (цены за 5 – 19, 20 – 49 и 50+ единиц).

- Заказы и их состав – таблицы Order и OrderItem, позволяющие хранить как общие сведения о заказе (клиент, адрес доставки, сумма, статус), так и детализированный перечень товаров с количеством и типом цены.

- Коммуникации – таблицы Chat и Message, реализующие связанный с заказом канал обмена сообщениями между клиентом и службой поддержки.

Для оптимизации производительности в базе данных предусмотрены индексы по наиболее часто используемым полям (категории товаров, статусы заказов, признак наличия на складе, признак администратора и т. д.), что ускоряет выполнение запросов выборки и упрощает построение аналитических отчетов. Связи реализованы по принципу «один-ко-многим» и «один-к-одному» там, где это обусловлено бизнес-логикой (например, один заказ – один чат поддержки).

В совокупности данная архитектура PostgreSQL-базы данных сочетает строгую структурированность реляционной модели с гибкостью работы с полуструктурированными данными, обеспечивает целостность и согласованность информации и позволяет безболезненно развивать систему за

счет добавления новых модулей и интеграций. Это делает её устойчивым фундаментом для дальнейшей автоматизации бизнес-процессов и расширения функциональности информационной системы.

2.2.3 Описание нормативно-справочной, входной и результатной информации

Информационное наполнение базы данных формируется из нескольких типов данных, каждый из которых играет свою роль в работе системы и обеспечивает полный цикл функционирования – от ввода первичных сведений до формирования аналитических отчетов.

Нормативно-справочная информация представлена стабильными, редко изменяемыми наборами данных, которые служат основой для классификации и унификации записей в операционных таблицах. К таким данным относятся:

- Категории товаров – фиксированные группы для систематизации ассортимента (например, конструкторы, куклы, настольные игры).
- Возрастные группы – диапазоны, по которым сегментируется целевая аудитория продукции.
- Материалы – перечень допустимых значений для описания состава товаров.
- Страны – справочник географического происхождения или поставки.
- Статусы заказов – последовательность состояний, описывающая этапы жизненного цикла заказа (от «Нового» до «Доставленного»).

Эти сведения обеспечивают целостность и корректность данных за счет ограничения возможных значений полей и используются в связях с основной операционной информацией.

Входная информация формируется в процессе работы пользователей и включает:

- Регистрационные данные – сведения, необходимые для создания учетной записи и идентификации пользователя, включая контактную и организационную информацию.
- Товарная информация – детализированные описания продукции, изображения, цены, наличие на складе.
- Заказы – сформированные клиентами заявки, включающие набор позиций, адрес доставки, условия оплаты и выбранные товары.
- Сообщения – переписка между пользователем и службой поддержки в рамках заказов, хранящаяся в системе чатов.

Этот блок данных является динамичным и постоянно пополняется в ходе эксплуатации системы.

Результатная информация формируется на основе обработки и анализа входных данных.

Таким образом, информационное обеспечение системы базируется на четком разграничении данных по их назначению, частоте изменения и источникам формирования, что способствует упорядоченной организации хранения, высокой скорости доступа и повышенной аналитической ценности информации.

2.3 Архитектура системы

2.3.1 Диаграмма компонентов

Клиентская часть приложения реализована на React с использованием TypeScript и организована по компонентной архитектуре. В её состав входят несколько ключевых групп:

- Страницы – верхнеуровневые контейнеры, отвечающие за маршруты и сборку интерфейса из отдельных компонентов:
 - Landing – главная страница с обзором ассортимента и промо-блоками.
 - Shop – каталог товаров с поиском, фильтрацией и добавлением в корзину.
 - Orders – личный кабинет с историей заказов и их статусами.
 - Admin – административная панель для управления товарами, заказами и пользователями.
- UI-компоненты – переиспользуемые элементы интерфейса:
 - Navbar – панель навигации с логотипом, поиском, меню и значком корзины.
 - ProductCard – карточка товара с изображением, ценой и кнопками действий.
 - CartModal – модальное окно корзины с возможностью изменить количество товаров и оформить заказ.
- Хуки для состояния – кастомные React-хуки, инкапсулирующие логику работы с данными и событиями:
 - useAuth – управление аутентификацией, токенами и профилем пользователя.
 - useCart – хранение и обновление данных корзины, расчёт суммы, синхронизация с сервером.

Эти модули взаимодействуют с сервером через HTTP API, используя такие библиотеки, как TanStack Query, для кэширования и синхронизации данных.

Серверная часть приложения реализована на Node.js с использованием Express и разделена на несколько слоёв:

- Routes (API Endpoints) – набор REST-маршрутов, которые принимают запросы от фронтенда и направляют их к соответствующей бизнес-логике:

- маршруты аутентификации (/api/auth/...),
- работы с товарами (/api/products/...),
- управления заказами (/api/orders/...),
- чатов (/api/chats/...),
- административных функций (/api/admin/...).

- Storage (Слой данных) – модуль доступа к базе данных PostgreSQL через ORM Drizzle. Отвечает за выполнение CRUD-операций с таблицами пользователей, товаров, заказов, чатов и сообщений.

- Middleware – промежуточные обработчики, интегрированные в конвейер Express:

- Аутентификация – проверка JWT-токенов и прав доступа.
- Логирование – запись информации о запросах и ответах для отладки и аудита.
- Обработка ошибок – централизованная генерация и отправка структурированных сообщений об ошибках клиенту.
- Обработка файлов – приём и сохранение загружаемых изображений и документов (Multer).

Такое разделение обеспечивает модульность, лёгкое масштабирование и изоляцию логики между слоями.

2.3.2 Диаграмма развертывания

Диаграмма развертывания иллюстрирует физическую и виртуальную инфраструктуру системы, показывая аппаратные и программные узлы, размещение артефактов и каналы связи без учёта платёжных шлюзов и модулей техподдержки. Ее можно увидеть на рисунке 2.10

Система развёрнута в трёх логических зонах:

- Пограничная (DMZ) с балансировщиками и публичным веб-проксем
- Прикладная зона с контейнерными кластерами
- Зона хранения данных и кэша

В DMZ располагаются два узла L4-балансировщика с активным режимом, каждый подключён к публичному IPv4 и IPv6 и настроен на TLS-терминацию (сертификаты Let's Encrypt). Балансировщики распределяют HTTPS-трафик на группу Web-Tier, в которую входят по 3 реплики Nginx-прокси (Docker-контейнеры). Они фильтруют HTTP-заголовки (CSP, HSTS),

выполняют первичную проверку IP-блэка списка и пробрасывают запросы внутрь виртуальной сети[20].

Ingress-контроллер (NGINXIngress) создаёт маршруты доступа по разным доменным именам (api.company.ru, app.company.ru) и обеспечивает ограничение скорости (rate-limiting) на уровне HTTP[21].

Зона хранения данных:

– PostgreSQL-кластер в режиме primary – sync-standby, развернутый через StatefulSet, PVC на Ceph-FS, ежедневный физический бэкап через Velero и диапазонное партиционирование таблицы traffic_stats по месяцам.

– Replit Sentinel-кластер из трёх мастеров с репликами для сессий и кэша, мониторинг задержек и автопереключение при отказе.

Сетевые связи:

– Все узлы подключены к виртуальному оверлей-сегменту.
– Между DMZ и прикладной зоной стоит межсетевой экран с глубоким анализом пакетов (IPS), запрещающим прямые подключения с публичного интернета к базе данных и кэшу.

CI/CD конвейер:

– при пуше в ветку main автоматически создаётся артефакт Replit образа (как Docker образ, но все автоматизировано), прогоняются unit- и integration-тесты, проверяется статический анализ кода (ESLint, SonarQube), затем производится деплой в staging через Helm.

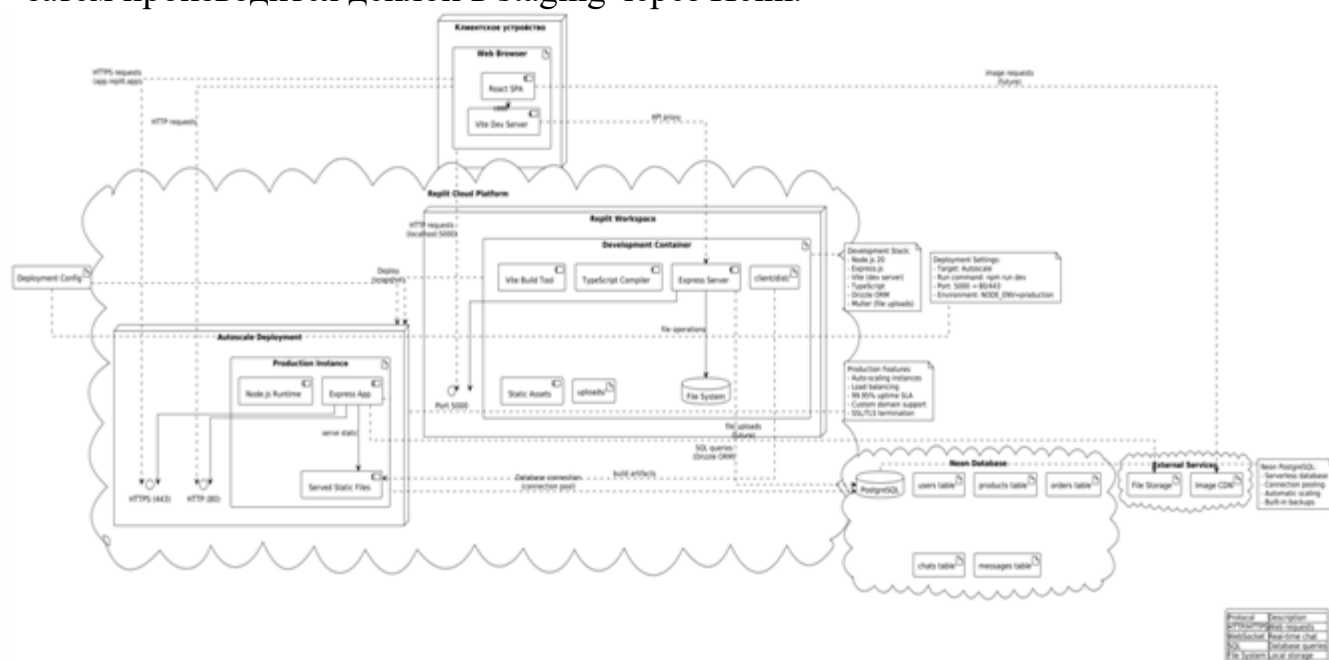


Рисунок 2.7 - Диаграмма развертывания.

Примечание – Источник – Собственная разработка.

2.4 Разработка программных модулей

2.4.1 Клиентский модуль

Клиентский модуль интернет-магазина «Семь Драконов» представляет собой одностраничное веб-приложение (SPA), реализованное с использованием библиотеки React в сочетании с современным инструментарием разработки, таким как TypeScript, Tailwind CSS и Shadcn UI. Архитектура построена на основе функциональных компонентов с применением как стандартных хуков React (useState, useEffect), так и кастомных, разработанных под потребности проекта (useAuth для управления аутентификацией, useCart для работы с корзиной, useToast для уведомлений). Такой подход обеспечивает высокую модульность кода, лёгкое тестирование отдельных частей и возможность масштабирования функционала без кардинальных изменений архитектуры. Компоненты и страницы разделены на логические группы: контейнеры верхнего уровня (Landing, Shop, Orders, Admin и специализированные административные панели), переиспользуемые UI-элементы (кнопки, формы, таблицы, модальные окна), а также специализированные блоки для отображения и взаимодействия с товарами (например, ProductCard, ProductModal, фильтры, корзина). Визуальная часть приложения стилизована при помощи Tailwind CSS и библиотеки Shadcn UI, что позволяет сочетать кастомный дизайн с готовыми доступными и адаптивными компонентами, а также сохранять единую дизайн-систему на всех страницах.

Взаимодействие с серверной частью осуществляется через REST API по протоколам HTTP/HTTPS, а для управления серверным состоянием и оптимизации работы с данными применяется библиотека TanStack Query. Она обеспечивает автоматическое кэширование, синхронизацию и поддержку оптимистичных обновлений, что делает интерфейс отзывчивым и минимизирует время ожидания пользователя. Все API-запросы инкапсулированы в сервисных модулях, что упрощает их повторное использование и обслуживание.

Разработка и сопровождение клиентского модуля ведутся в облачной среде Replit, которая предоставляет интегрированную веб-IDE, контейнеризированную рабочую среду, встроенный терминал, поддержку Git и мгновенный доступ к приложению по публичному URL. Использование Replit позволяет полностью избежать необходимости локальной установки и настройки инструментов: весь стек – от React и Vite Dev Server в режиме разработки до Node.js и Express в бэкенде – разворачивается в едином контейнере. В процессе разработки React-приложение запускается через Vite Dev Server с функцией горячей перезагрузки (HMR), что позволяет видеть изменения мгновенно без полной перезагрузки страницы, а статические

ресурсы и итоговая сборка хранятся в директории `client/dist` и папке `uploads` внутри файловой системы контейнера. При деплое в продакшн Replit автоматически разворачивает приложение на Autoscale-инстансах с балансировкой нагрузки, SSL/TLS-терминацией, поддержкой пользовательских доменов и высокой доступностью (SLA 99, 95%). Благодаря этому разработчики могут быстро вносить изменения, тестировать их на реальных данных и тут же предоставлять заказчику или тестовой группе для оценки, а интеграция фронтенда и бэкенда внутри одной среды упрощает отладку и исключает проблемы, связанные с несовместимостью окружений. Такой подход сочетает в себе преимущества облачной разработки и мгновенного развертывания, обеспечивая высокую скорость итераций и устойчивое качество продукта.

2.4.2 Серверный модуль

Серверный модуль интернет-магазина «Семь Драконов» построен на основе Node.js с использованием фреймворка Express.js и написан на TypeScript, что обеспечивает строгую типизацию, облегчает рефакторинг и минимизирует вероятность логических ошибок на ранних этапах разработки. Архитектура сервера организована по модульному принципу: логика разделена на независимые блоки маршрутов, каждый из которых отвечает за конкретную доменную область – авторизация и регистрация пользователей, управление каталогом товаров, оформление и обработка заказов, работа с чатом, а также административные операции. Каждый такой маршрутный модуль инкапсулирует контроллеры для обработки HTTP-запросов, обращается к сервисному слою для выполнения бизнес-логики и использует слой доступа к данным для взаимодействия с базой. Эта изоляция зон ответственности упрощает поддержку и добавление нового функционала, исключая нежелательные побочные эффекты.

Важным элементом серверной архитектуры является система промежуточных обработчиков (middleware). Она включает:

- аутентификацию и авторизацию с использованием JWT-токенов, позволяющую разграничивать доступ к защищённым ресурсам;
- централизованную обработку ошибок, формирующую единообразные структурированные ответы для клиента и скрывающую внутренние детали реализации;
- логирование запросов и ответов, необходимое для мониторинга и аудита;
- модуль для загрузки и сохранения файлов на основе библиотеки Multer, который принимает изображения товаров и другие ресурсы, проверяет их и

сохраняет в специально выделенную директорию в файловой системе контейнера;

– обработчик статических ресурсов, который позволяет Express напрямую отдавать изображения, стили и скрипты без лишней нагрузки на бизнес-логику.

Интеграция с базой данных реализована через ORM Drizzle, которая соединяется с облачной PostgreSQL-инстанцией на платформе Neon. Это решение обеспечивает декларативное описание схем таблиц на TypeScript, позволяет использовать статическую проверку запросов, минимизировать расхождения между кодом и структурой данных, а возможности Neon (серверлесс-архитектура, автоскейлинг, connection pooling, резервное копирование) делают работу с данными отказоустойчивой и масштабируемой. Для загрузки конфигураций и учётных данных используются переменные окружения, что позволяет легко адаптировать сервер к разным средам.

Отдельного внимания заслуживает то, что весь серверный модуль – от разработки до деплоя – живёт в облачной среде Replit, совместно с клиентским приложением. На этапе разработки Express-сервер запускается в контейнере Replit Workspace, используя проброс порта (по умолчанию 5000), что даёт возможность тестировать API локально, но в облаке. При этом сервер и фронтенд работают в одном окружении, что убирает проблемы с CORS и разными конфигурациями между dev и prod. Разработчик получает доступ к полноценной веб-IDE с автодополнением, встроенным терминалом, поддержкой Git и моментальным предпросмотром работы сервера в браузере. Использование Vite Dev Server в том же окружении позволяет одновременно вести работу над клиентом и сервером, а встроенный прокси перенаправляет запросы SPA к API без дополнительной настройки.

При деплое Replit выполняет автоматическую упаковку проекта и развёртывает его на Autoscale-инстансах. Это среда продакшн-класса с балансировкой нагрузки, возможностью горизонтального масштабирования, 99, 95% SLA по доступности, SSL/TLS-терминацией и поддержкой собственных доменов. Переход из режима разработки в продакшн в Replit происходит без смены окружения и инфраструктуры, что гарантирует идентичность условий и предотвращает неожиданные ошибки. Кроме того, Replit хранит как исходный код, так и артефакты сборки, а статические файлы и загруженные ресурсы могут обслуживаться напрямую из контейнера. Для администратора это означает, что обновление серверной логики или конфигураций требует лишь заливки новых изменений в репозиторий или внесения правок в IDE, после чего Replit автоматически пересобирает и перезапускает приложение.

Таким образом, серверный модуль «Семь Драконов» – это не просто набор Express-маршрутов и middleware, а полнофункциональный backend-сервис с чёткой модульной структурой, безопасным доступом к данным, развитой системой обработки запросов и ошибок, интеграцией с современными облачными технологиями хранения и развёрнутый в среде, которая объединяет разработку, тестирование и эксплуатацию в едином облачном цикле.

2.4.3 Модуль работы с данными

Модуль работы с данными интернет-магазина «Семь Драконов» реализован с использованием современной ORM-библиотеки Drizzle, обеспечивающей полную типобезопасность за счёт декларативного описания схем базы данных на языке TypeScript. Такой подход позволяет интегрировать структуру таблиц непосредственно в код приложения, получая строгую проверку типов уже на этапе компиляции и исключая целый класс ошибок, связанных с несоответствием между схемой данных и бизнес-логикой. Каждая таблица (пользователи, товары, заказы, чаты, сообщения) описывается в виде отдельного модуля схемы, а связи между ними формализуются через ключи и ограничения, что делает навигацию по данным предсказуемой и прозрачной.

Подключение к базе данных PostgreSQL, размещённой на облачной платформе Neon, осуществляется через пул соединений, что позволяет эффективно обслуживать большое количество параллельных запросов и избегать затрат на частое открытие и закрытие подключений. Neon, как серверлесс-решение, автоматически масштабируется под нагрузку, а пуллинг соединений в связке с Drizzle гарантирует стабильную работу даже при пиковых обращениях. Для формирования, модификации и контроля структуры базы данных используется инструмент Drizzle Kit, который обеспечивает автоматическую генерацию миграций, их версионирование и безопасное применение в разных средах (разработка, тест, продакшн). Все миграции хранятся в репозитории вместе с кодом, что позволяет легко отслеживать историю изменений и поддерживать согласованность структуры БД между членами команды.

Взаимодействие с данным модулем происходит через слой абстракции Storage на стороне сервера: обработчики API не обращаются напрямую к SQL, а используют типобезопасные запросы Drizzle, что повышает читаемость и сопровождаемость кода. Благодаря этому добавление новых полей или таблиц не требует ручного переписывания SQL-запросов, а статическая типизация гарантирует, что изменения будут корректно отражены во всех местах

использования. Такой подход облегчает рефакторинг и ускоряет разработку новых функций.

Разработка и эксплуатация этого модуля интегрированы в среду Replit, совместно с клиентским и серверным кодом. В режиме разработки соединение с базой Neon осуществляется через защищённые параметры окружения, заданные в конфигурации Replit Workspace, а Drizzle Kit позволяет генерировать и применять миграции прямо из онлайн-IDE, без необходимости локальной установки PostgreSQL. Это упрощает онбординг новых разработчиков и даёт возможность мгновенно тестировать изменения схемы в общем контейнере. При деплое в продакшн та же конфигурация используется Autoscale-инстансом, что гарантирует идентичность окружений и исключает ошибки, связанные с несовпадением версий схемы базы данных.

В совокупности, модуль работы с данными представляет собой надёжный, гибкий и легко масштабируемый слой хранения, который сочетает преимущества современного ORM-подхода, облачных технологий Neon и контейнеризированной инфраструктуры Replit, обеспечивая стабильную работу интернет-магазина и высокую скорость разработки новых функций.

2.5 Технологическое обеспечение разработки

2.5.1 Процесс тестирования

Процесс тестирования в проекте опирается на встроенные механизмы статического анализа и проверки качества кода. Использование TypeScript обеспечивает строгую проверку типов на этапе компиляции, что позволяет выявлять потенциальные ошибки ещё до запуска приложения. Для поддержания единых стандартов оформления и предотвращения стилистических и логических огрехов применяются инструменты ESLint (анализ и подсветка проблемных конструкций) и Prettier (автоматическое форматирование кода в соответствии с заданным стилем). Такая комбинация инструментов позволяет автоматически контролировать качество кода на всех этапах разработки и снижает вероятность появления ошибок в рабочей сборке.

2.5.2 Процесс развертывания

Процесс развертывания интернет-магазина «Семь Драконов» организован таким образом, чтобы обеспечить единый, автоматизированный и воспроизводимый цикл публикации приложения из среды разработки в production-окружение. Фронтенд-часть, реализованная на React, проходит этап сборки с помощью высокопроизводительного инструментального набора Vite. На этом шаге исходный TypeScript-код и JSX-шаблоны транслируются в

оптимизированный JavaScript, применяются минификация и tree-shaking для удаления неиспользуемого кода, формируются статические ресурсы (HTML, CSS, изображения) и подготавливается структура финальной сборки в директории dist. Параллельно выполняется обработка и упаковка CSS-стилей, а также встроенных ассетов, что позволяет сократить время загрузки и повысить производительность клиентского приложения.

Серверный модуль, разработанный на Express.js с использованием TypeScript, компилируется при помощи esbuild – сверхбыстрого компилятора и бандлера, который преобразует TypeScript-файлы в исполняемый JavaScript-код, объединяет модули и выполняет оптимизацию для работы в среде Node.js.. Такой подход минимизирует время сборки и гарантирует единообразие результата на всех средах.

После завершения этапов фронтенд- и бэкенд-сборки осуществляется упаковка итоговых артефактов в единое приложение, готовое к запуску. В production-режиме Express-сервер обслуживает не только API-эндпоинты, но и статические файлы фронтенда, что упрощает архитектуру и устраняет необходимость в отдельном веб-сервере. Это обеспечивает согласованное поведение приложения и минимизирует количество точек отказа.

Весь процесс интегрирован в облачную платформу Replit, которая объединяет в одном контейнере как среду разработки, так и окружение для продакшн-развёртывания. Разработчик, работая в Replit Workspace, запускает сборку фронтенда и бэкенда напрямую из онлайн-IDE, а затем инициирует деплой на Autoscale Deployment. Это окружение автоматически масштабируется под нагрузку, балансирует входящий трафик, обеспечивает высокую доступность (SLA 99, 95 %), поддержку пользовательских доменов и SSL/TLS-шифрования. Перед деплоем Replit применяет конфигурацию окружения, определённую в файле настроек проекта (порт, команда запуска, переменные окружения), что гарантирует воспроизводимость и предотвращает несовместимости между dev- и prod-средами.

Таким образом, развёртывание превращается в последовательность шагов, полностью автоматизированных и исполняемых в одной инфраструктуре – от генерации статических ресурсов и компиляции серверной логики до публикации готового приложения на масштабируемой облачной платформе, доступной пользователям по защищённому адресу.

2.5.3 Мониторинг и логирование

В рамках серверного модуля интернет-магазина «Семь Драконов» реализован встроенный механизм мониторинга и логирования, призванный обеспечить прозрачность работы приложения, упростить отладку и повысить качество обслуживания пользователей. Логирование охватывает весь цикл обработки API-запросов: фиксируется время получения запроса, используемый HTTP-метод, целевой эндпоинт, идентификатор пользователя (при наличии аутентификации), параметры запроса и тело (для допустимых типов данных). Для каждого запроса дополнительно вычисляется время выполнения – это позволяет оперативно выявлять «узкие места» в производительности и отслеживать их динамику.

Ответы сервера также регистрируются в логах с указанием кода HTTP-статуса, кратким описанием результата и возможными диагностическими комментариями. Такая двусторонняя фиксация – запроса и ответа – даёт целостную картину работы системы и облегчает воспроизведение проблемных сценариев.

Обработка ошибок построена по централизованной схеме: любое исключение, возникающее на этапе маршрутизации, исполнения бизнес-логики или обращения к базе данных, перехватывается единым middleware-компонентом.

Логи структурируются в формате, удобном для анализа и автоматической обработки, и могут быть дополнительно направлены в внешние системы мониторинга или храниться в облачных сервисах для долгосрочного анализа. Такой подход позволяет не только оперативно реагировать на сбои, но и формировать метрики доступности, времени отклика и частоты возникновения ошибок, что напрямую влияет на качество пользовательского опыта и стабильность работы приложения.

ГЛАВА 3

РЕАЛИЗАЦИЯ, ТЕСТИРОВАНИЕ И ЭКСПЛУАТАЦИЯ

3.1 Руководство по установке и настройке

3.1.1 Системные требования

Вся серверная и клиентская часть веб-приложения интернет-магазина развёрнута и функционирует в облачной среде Replit, что обеспечивает мгновенный доступ к рабочей среде, автоматическое обновление кода и независимость от локальной инфраструктуры.

Технологический стек

- Backend: Node.js (Express.js для REST API), модульная структура с выделенными слоями маршрутизации, бизнес-логики и доступа к данным.
- Frontend: React.js с компонентным подходом, интеграция с API через axios/fetch, адаптивная верстка (HTML5, CSS3, Flexbox/Grid).
- База данных: PostgreSQL (облачная база Neon/Postgres в контейнере Replit), поддержка транзакций и индексов.
- Системы логирования и мониторинга: встроенные middleware-модули + возможность подключения внешних сервисов через API-ключи.
- Контейнеризация: использование встроенных Replit-сред для изоляции окружения и управления зависимостями через package.json.
- Управление версиями: Git-репозиторий, встроенный в Replit, с автоматическим деплоем при коммитах.

Требования к рабочей среде

- Сервер: виртуальный контейнер Replit с предустановленным Node.js (v16+), доступом к облачной базе данных и выделенными ресурсами (CPU/RAM в зависимости от тарифного плана Replit).
- Клиентское окружение: любой современный веб-браузер с поддержкой ES6+ (Chrome, Firefox, Edge, Safari).
- Сетевое подключение: устойчивое интернет-соединение для работы с API и WebSocket-каналами.

Преимущества развёртывания на Replit

- Автоматическая сборка и перезапуск приложения при изменениях.
- Единая среда для разработки, тестирования и эксплуатации.
- Возможность совместной работы над проектом в реальном времени.

3.1.2 Пошаговая инструкция развёртывания

Ниже приведён регламент запуска серверной и клиентской части интернет-магазина в облачной среде Replit. Он ориентирован на воспроизводимое, документированное и удобное развёртывание, подходящее как для среды разработки, так и для эксплуатации в production.

1. Клонирование и подготовка проекта

- Откройте проект в рабочем пространстве Replit (через Git-репозиторий или импорт ZIP-архива).

- Убедитесь, что в `replit` и `replit.nix` корректно указана команда запуска.

2. Установка зависимостей

- Установить все зависимости, указанные в `package.json` (backend и frontend).

- При использовании Replit установка выполняется в изолированной контейнерной среде.

3. Настройка переменных окружения

- В панели Replit Secrets или в файле `.env` создайте ключ

- Для Neon/PostgreSQL подключение автоматическое

4. Создание и синхронизация схемы базы данных

`npm run db: push`

- Команда синхронизирует модель данных с реальной БД, создавая таблицы и индексы.

- Логи операции помогают убедиться, что миграция прошла без ошибок.

5. Запуск приложения

- Сервер автоматически перезапускается при изменениях в коде.

- Логи и ошибки выводятся в консоль Replit.

- Сначала выполняется сборка оптимизированного frontend-кода (React).

- Затем запускается Node.js-сервер, обслуживающий статические файлы и API.

6. Верификация запуска

- Перейдите по URL Replit-проекта (формат `https://<project>.<username>.repl.co`).

- При необходимости – просмотрите логи для проверки подключения к базе и корректности работы middleware.

3.1.3 Настройка окружения и интеграций

Данный раздел описывает параметры конфигурации облачного окружения Replit, интеграцию с базой данных Neon PostgreSQL, а также организацию файлового хранилища и систему переменных окружения для корректной работы приложения.

1. Конфигурация NeonPostgreSQL

- Создание БД:

1. Зарегистрируйтесь в Neon.tech и создайте проект с типом базы PostgreSQL.

2. Определите параметры подключения (host, порт, имя БД, имя пользователя, пароль, SSL).

- Строка подключения: Формат:

```
postgresql: //<user>: <password>@<host>/<dbname>?sslmode=require
```

Необходимо скопировать и сохранить в переменную окружения DATABASE_URL.

- Интеграция с приложением:

- В ReplitSecrets создайте ключ DATABASE_URL и вставьте полученную строку.

- Убедитесь, что используемый ORM/драйвер настроен на чтение этой переменной.

- Выполните синхронизацию структуры командой: `npm run db: push`

2. Настройка файловых загрузок

- Директория: в корне проекта создаётся папка. Она используется для временного хранения загружаемых изображений и документов.

- Конфигурация Express:

```
import path from 'path';
import multer from 'multer';
const storage = multer.diskStorage({
  destination: path.join(process.cwd(), 'uploads'),
  filename: (req, file, cb) => {
    cb(null, Date.now() + '-' + file.originalname);
  }
});
```

```
export const upload = multer({ storage });
```

- Безопасность и доступ:

- Закройте прямой публичный доступ на уровне роутинга, раздавая файлы только авторизованным пользователям.

- При деплое на Replit учтите, что файловая система – `hemeral`, т.е. данные не сохраняются при перезапуске. Для персистентного хранения можно подключить облачные S3-совместимые хранилища.

3. Настройка переменных окружения

В проекте используются конфигурационные параметры, вынесенные в переменные окружения.

Обязательные переменные (добавляются в ReplitSecrets):

DATABASE_URL=<строка-подключения-Neon-PostgreSQL>

PORT=3000

NODE_ENV=development # или production

UPLOADS_DIR=uploads

JWT_SECRET=<секрет-для-токенов>

3.2 Описание контрольного примера

3.2.1 Сценарий использования системы

Ниже – полный жизненный цикл работы корпоративного клиента с ИС «Семь Драконов»: от регистрации компании до оформления и сопровождения заказа, общения через чат и отслеживания статуса. Сценарии дополнены административной стороной процесса для целостной картины.

Пользовательский сценарий (полный цикл)

1. Регистрация компании
 - Действие: Переход на главную страницу, выбор «Регистрация».
 - Ввод данных: E-mail организации, название, юридический адрес, контактное лицо, телефон.
 - Верификация: Письмо с ссылкой подтверждения; опционально – ручная модерация реквизитов администратором.
 - Результат: Создан корпоративный аккаунт.
2. Авторизация
 - Действие: Вход по e-mail и паролю или SSO (если подключено).
 - Результат: Доступ к каталогу с персональными ценами и лимитами.
3. Просмотр каталога
 - Действие: Навигация по категориям игрушек, фильтры (возраст, бренд, маржа, остатки), поиск.
 - Информация: Отображаются оптовые цены с учётом прайс-уровня, доступные остатки, минимальные партии, сроки поставки.
 - Результат: Сформирован список интересующих SKU.
4. Добавление в корзину
 - Действие: Выбор SKU, указание количества с проверкой минимальной партии и кратности упаковки.
 - Проверки: Резервы на складе, ограничения по лимиту кредита, валюта и налоговые условия.

- Результат: Корзина с промежуточным итогом, расчёт скидок/акций, прогноз даты отгрузки.

5. Оформление заказа

- Данные: адрес, контакт, комментарии.
- Документы: Автоматическая генерация счёта, договора/спецификации; при предоплате – реквизиты оплаты.

- Результат: Создан заказ с номером, статус «Новый».

6. Общение через чат

- Действие: Переписка с менеджером: уточнение ассортимента, замены, сроки поставки, согласование цены.

- Функции: быстрые ответы; протоколирование в карточке заказа.

- Результат: Заказ актуализирован, зафиксированы договорённости.

7. Отслеживание статуса

- Действие: Просмотр таймлайна заказа в личном кабинете.

- Статусы: Новый → В обработке → Подтверждён → На сборке → Отгружен → Доставляется → Завершён (или Возврат/Отмена).

- Уведомления: E-mail/чат о смене статуса, трекинг-номер ТК, закрывающие документы после доставки.

Личный кабинет клиента

- Обзор: открытые заказы, последние действия.

- Каталог и заказы: Корзины, быстрый повтор закупки по шаблонам, история заказов с фильтрами и экспортом.

- Чат: Канал с закреплением сообщений по заказу.

Административные функции

- Каталог и цены: Управление SKU, остатками, минимальными партиями, прайс-уровнями, акциями, комплектами.

- Заказы: Очередь заказов, назначение ответственных, изменение статусов, частичные отгрузки/допоставки.

- Аналитика: Продажи по категориям/клиентам, оборачиваемость, отказоустойчивость, SLA по поддержке.

Статусы заказа и типовые переходы

- Основные: Новый → В обработке → На сборке → Отгружен → Доставляется → Завершён.

- Альтернативные ветки: Новый → Отмена (клиент/система); На сборке → Частичная отгрузка; Доставляется → Возврат/Рекламация.

- Правила: Изменение статусов доступно согласно ролям; ключевые изменения фиксируются в журнале аудита и видимы в таймлайне заказа.

Сценарии по категориям клиентов

- Малый бизнес: Быстрая регистрация, стандартные партии, самовывоз или городская доставка, базовая поддержка через чат.
- Средний/крупный опт: Индивидуальные цены, кредитный лимит, согласование спецификаций, приоритетная сборка и персональный менеджер.
- Сетевые клиенты/маркетплейсы: Шаблоны заказов, паллетные партии, EDI/CSV-обмен, SLA на отгрузку и возвраты.
- Образовательные/соц. учреждения: Закупки по счёту, полный пакет закрывающих документов, расширенные поля для договоров и актов.

3.2.2 Демонстрация работы ключевых функций

При открытии приложения, нас встречает главная страница с описанием преимуществ Компании, их услуг и тд. Изображено на рисунке 3.1

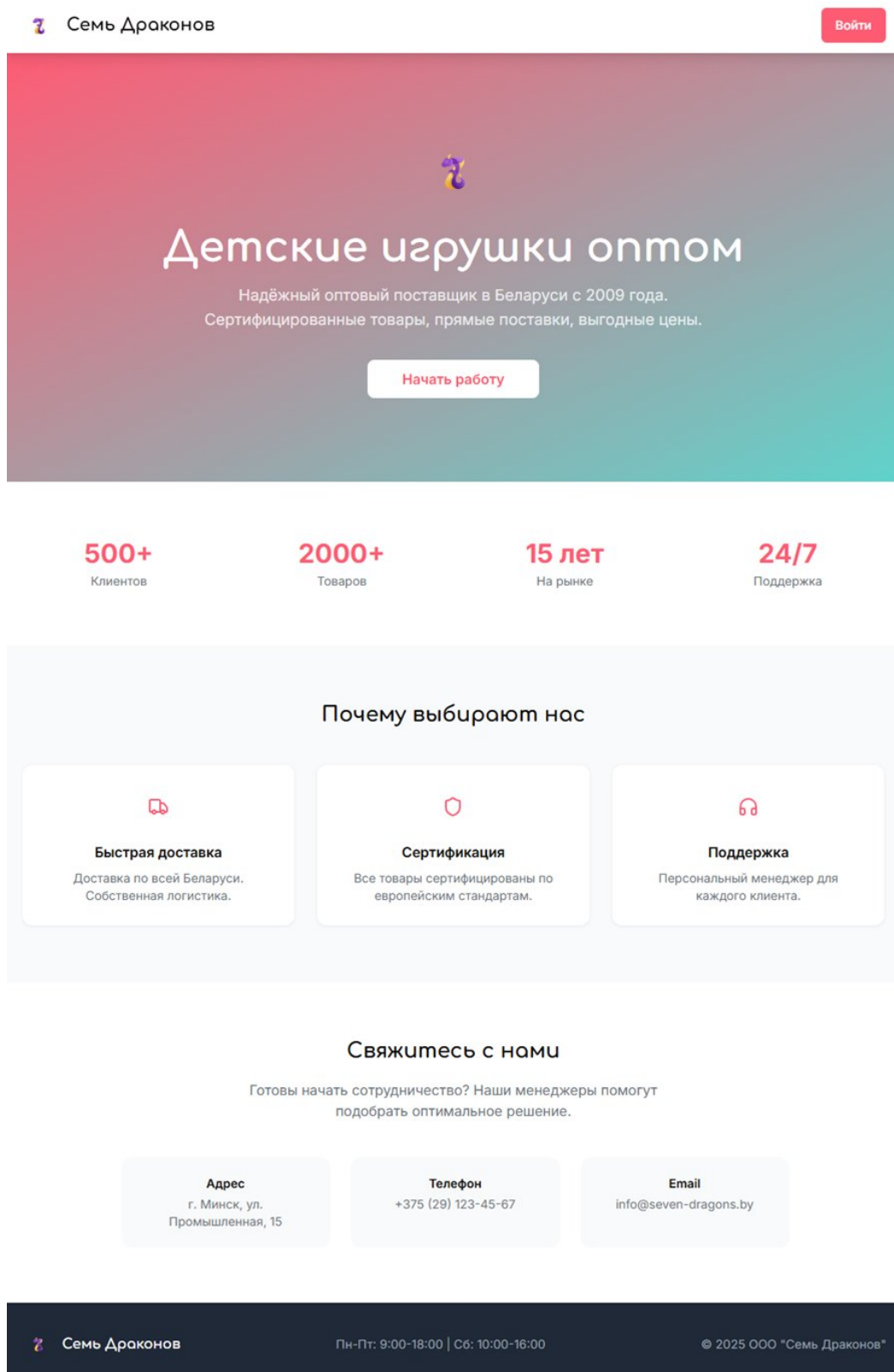


Рисунок 3.1 – Главная страница.

Примечание – Источник – Собственная разработка.

В верхнем меню есть навигационная панель, где можно найти кнопку Войти, при нажатии на которую перенесет на страницу авторизации. На рисунке 3.2 изображена страница авторизации.

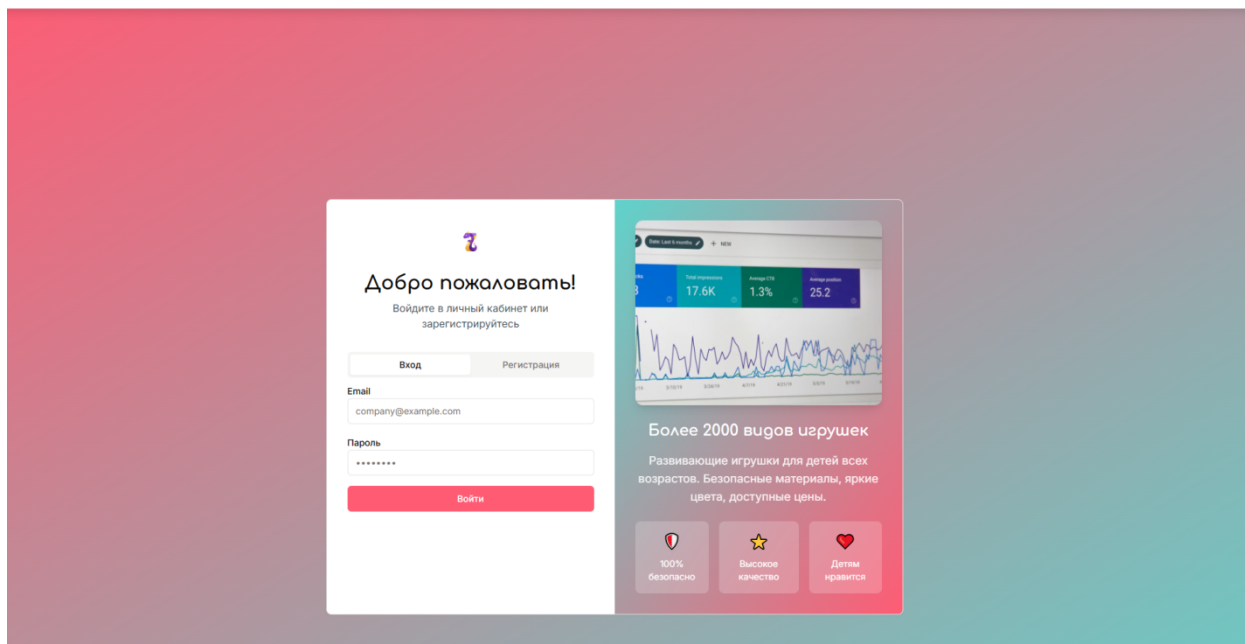


Рисунок 3.2 – Страница авторизации.

Примечание – Источник – Собственная разработка.

На рисунке 3.3 изображена страница, где описан каталог продукции компании, где так-же есть удобная система поиска, фильтрации и возможность добавить товары в корзину.

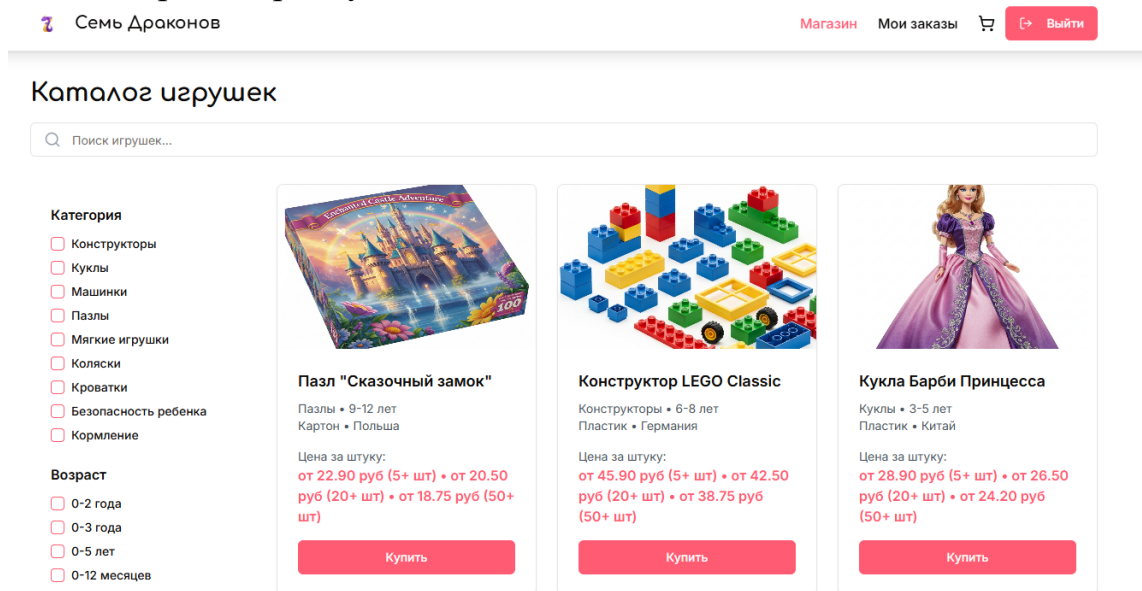


Рисунок 3.3 – Страница готовых решений у существующих Компаний.

Примечание – Источник – Собственная разработка.

На рисунке 3.4 изображено модальное окно корзины пользователя, где пользователь может просмотреть, что добавил, указать адрес и заказать окончательно товары.

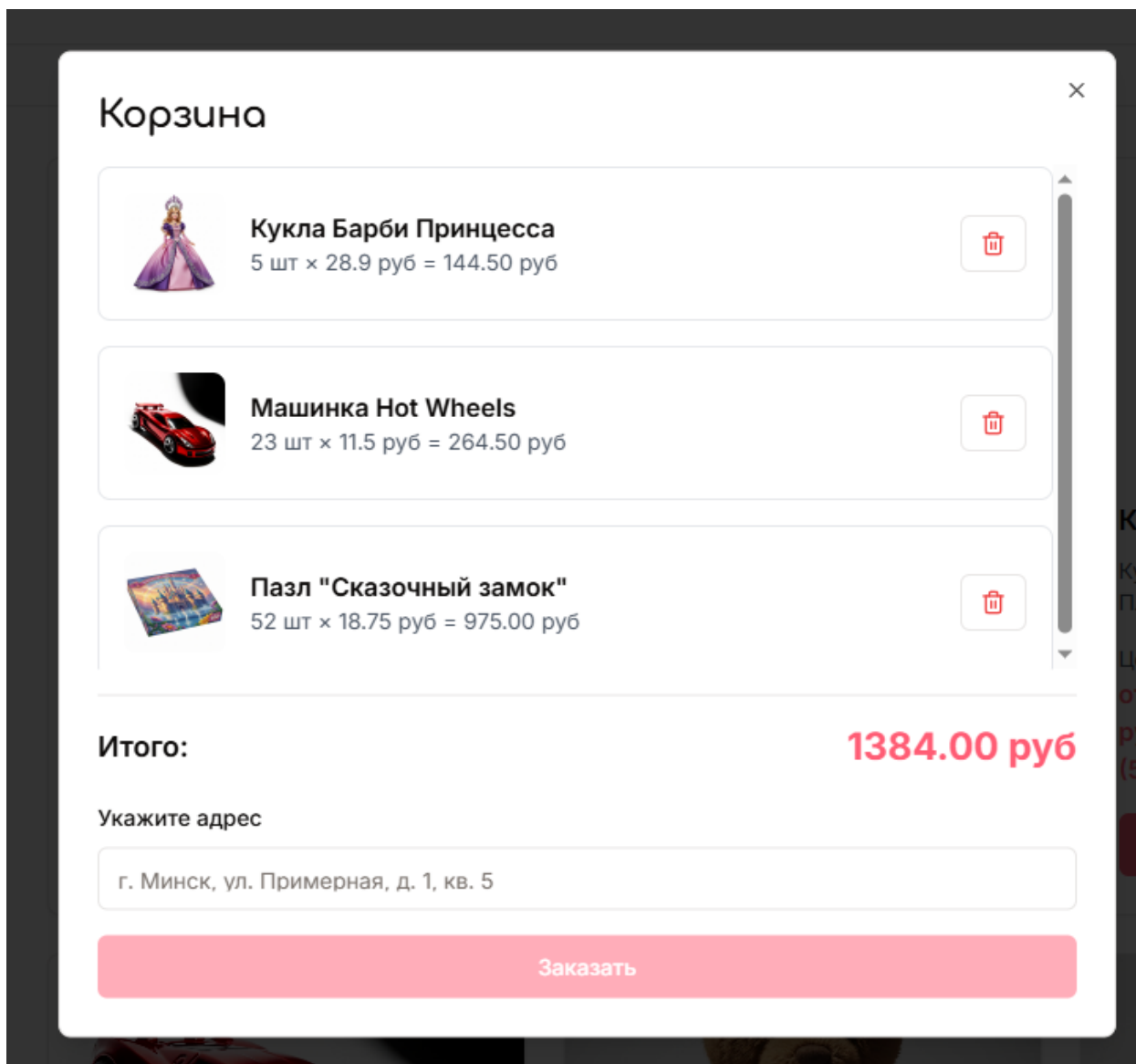


Рисунок 3.4 – Модальное окно корзины.

Примечание – Источник – Собственная разработка.

На рисунке 3.5 изображена страница, где пользователь может просмотреть все заказы, которые он когда либо заказывал.

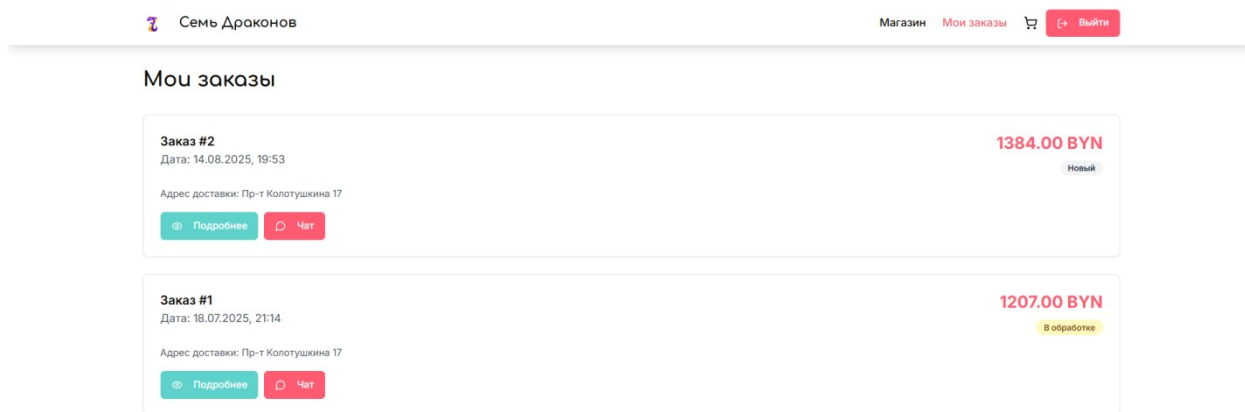


Рисунок 3.5 – Страница заказов пользователя.

Примечание – Источник – Собственная разработка.

На рисунке 3.6 изображено модальное окно, которое появляется при нажатии кнопки подробнее в заказе.

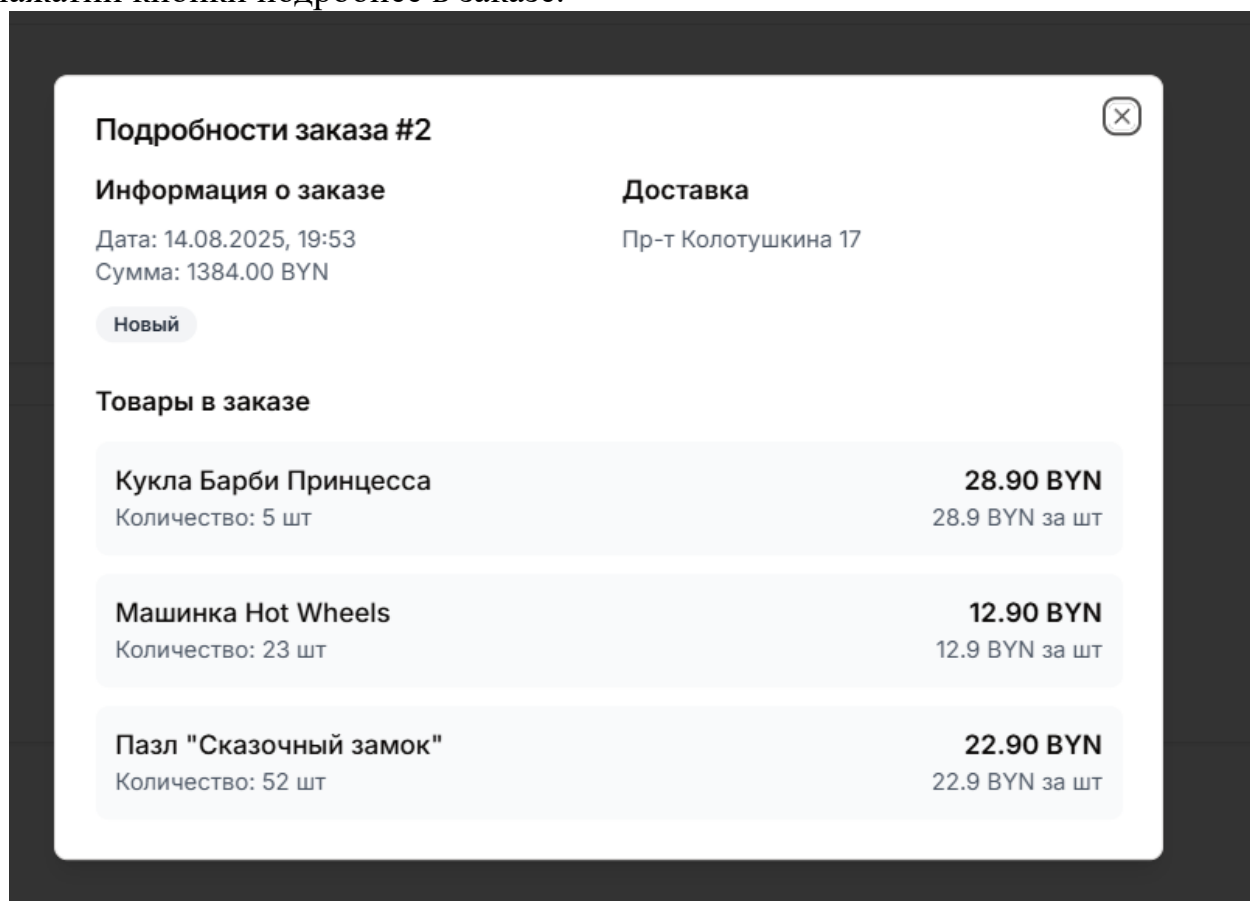


Рисунок 3.6 – Модальное окно подробностей заказа.

Примечание – Источник – Собственная разработка.

Так же, как показано на рисунке 3.7, пользователь может вести переписку с администратором, чтобы знать или уточнить разные детали.

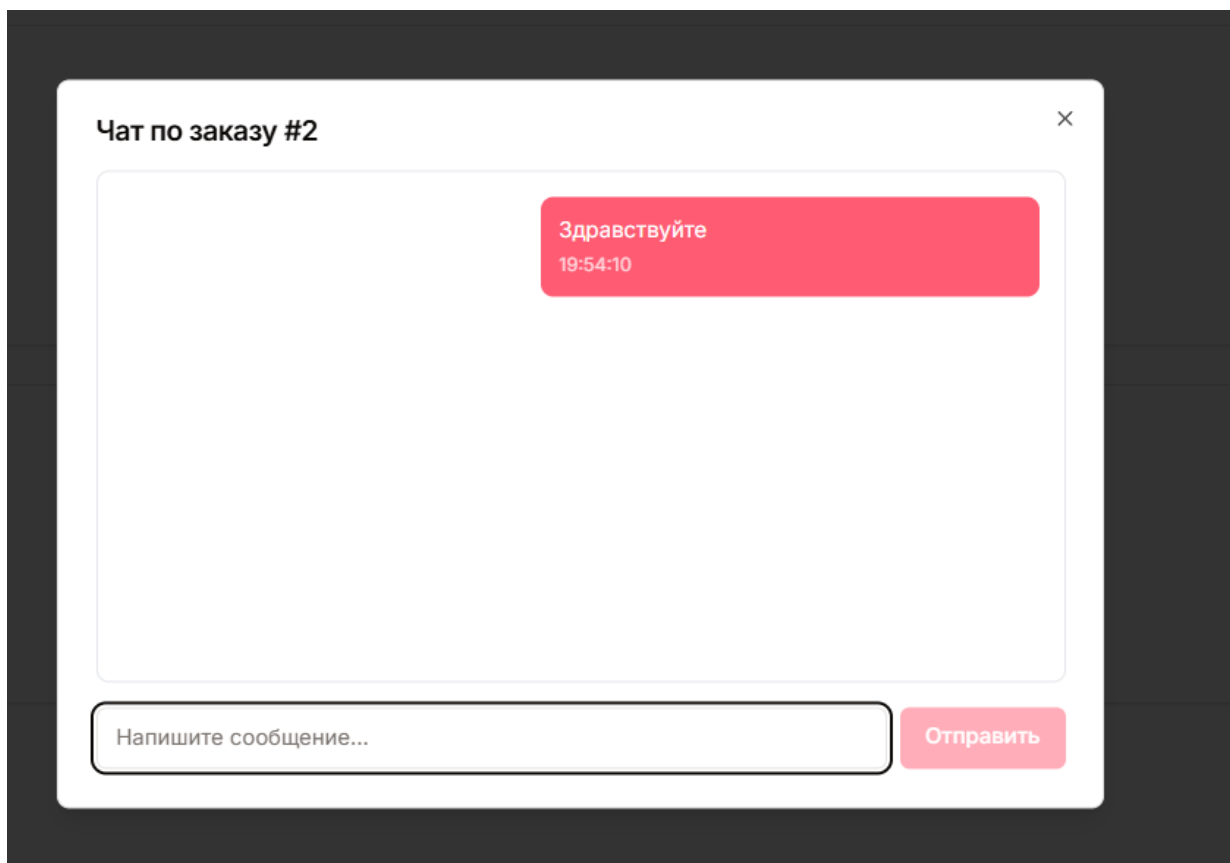


Рисунок 3.7 – Модальное окно чата с администратором.

Примечание – Источник – Собственная разработка.

У администратора же есть другие функции. Одна из возможностей, это просмотр всех продуктов, их редактирование, удаление или добавление новых. Показано на рисунках 3.8 и 3.9.

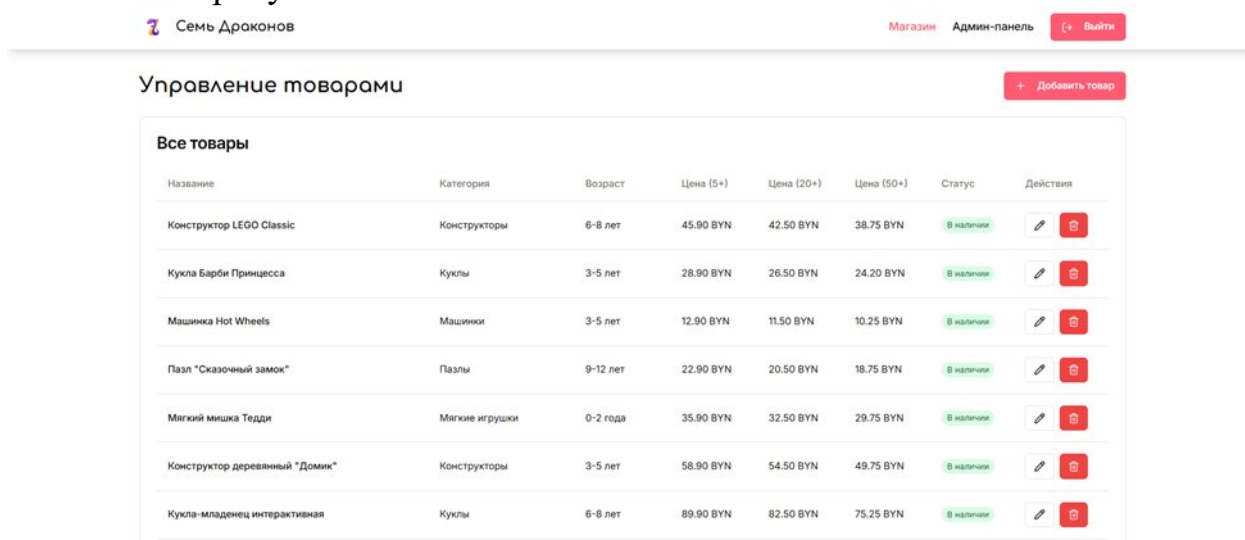


Рисунок 3.8 – Страница таблицы товаров.

Примечание – Источник – Собственная разработка.

Редактировать товар

Название:

Категория:

Описание:

Возрастная группа:

Материал:

Страна:

Цена (5+ шт):

Цена (20+ шт):

Цена (50+ шт):

URL изображения:

☒ В наличии

Рисунок 3.9 – Модальное окно редактирования товара.

Примечание – Источник – Собственная разработка.

На рисунке 3.10 открыта вкладка админ-панели, где администратор может ознакомиться с статистикой сайта и просмотреть все заказы и управлять их статусами.

Статистика и заказы

Всего пользователей: 2

Всего заказов: 2

Общая выручка: 2591.00 BYN

Активные заказы: 2

Заказы по статусам

1 Новый

1 В обработке

Все заказы

№ заказа	Дата	Клиент	Товары	Сумма	Статус	Действия
#2	14.08.2025, 19:53	ID: 1	Кукла Барби Принцесса x5 Машинка Hot Wheels x23 Пазл "Сказочный замок" x52	1384.00 BYN	Новый	
#1	18.07.2025, 21:14	ID: 1	Конструктор LEGO Classic x25 Кукла Барби Принцесса x5	1207.00 BYN	В обработке	В обработке

Рисунок 3.10 – Страница статистики сайта и заказов пользователей.

Примечание – Источник – Собственная разработка.

Так же администратор может просматривать пользователей, их статистику и ввести переписки с заказчиками. Все показано на рисунках 3.11 и 3.12

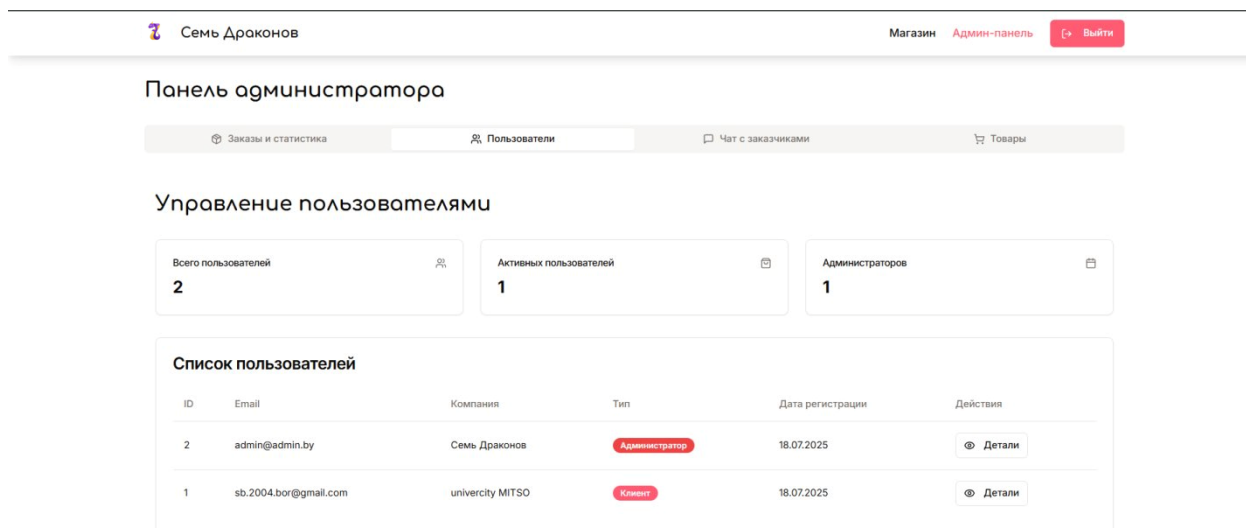


Рисунок 3.11 – Вкладка документов Компании.

Примечание – Источник – Собственная разработка.

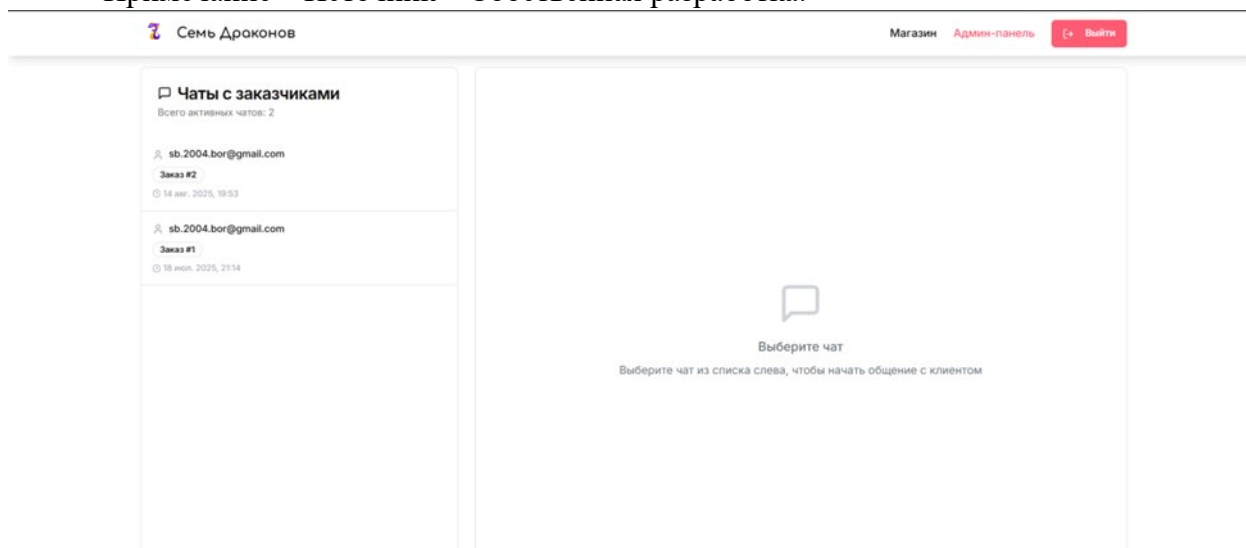


Рисунок 3.12 – Открытие одного из документов.

Примечание – Источник – Собственная разработка.

3.3 Результаты тестирования

3.3.1 Тест

Для фронтенда сначала были разработаны юнит-тесты, которые проверяли базовую логику форм авторизации и регистрации: корректное поведение при вводе валидных и невалидных данных, отображение ошибок на пустые поля. Интеграционные тесты убедились, что при взаимодействии с реальным API загружаются списки услуг и истории заказов, создаются новые заявки, пополняется баланс и обмениваются сообщениями тикетов поддержки.

Параллельно проводилось UI-тестирование, в котором сценарии навигации между разделами.

Для бэкенда набор юнит-тестов охватывал каждый REST-эндпоинт аутентификации и сервисных маршрутов: проверялась корректность HTTP-ответов при удачных и ошибочных запросах, работа middleware для сессий и валидация данных через Zod-схемы. Интеграционные тесты воспроизводили полный цикл «запрос – транзакция – ответ» с реальной базой данных, включая создание пользователей и заказов, обновление баланса, ведение истории статусов и генерацию сертификатов. Также был проверен отказоустойчивый откат при неудачных транзакциях и защищённость от SQL-инъекций благодаря Drizzle ORM.

Таблица 3.1 – Тест-кейсы клиентской части

ID	Модуль	Название тест-кейса	Предусловия	Шаги выполнения	Ожидаемый результат	Критерий
ТС_F001	Аутентификация	Регистрация нового пользователя	Пользователь не зарегистрирован	1. Открыть страницу регистрации 2. Заполнить все обязательные поля 3. Загрузить логотип 4. Нажать "Зарегистрироваться"	Пользователь успешно зарегистрирован и перенаправлен в магазин	Отображается уведомление об успешной регистрации
ТС_F002	Аутентификация	Авторизация с валидными и данными	Пользователь зарегистрирован	1. Открыть страницу входа 2. Ввести email и пароль 3. Нажать "Войти"	Пользователь авторизован и перенаправлен в каталог	Navbar показывает имя компании пользователя
ТС_F003	Каталог товаров	Просмотр списка товаров	Пользователь авторизован	1. Перейти на страницу "Магазин" 2. Дождаться загрузки товаров	Отображается список всех доступных товаров	Показаны карточки и товаров с изображениями и ценами
ТС_F004	Фильтрация	Фильтрация по категории	Открыта страница магазина	1. Открыть боковую панель	Отображаются только товары	Количество товаров

				фильтров 2. Выбрать категорию "Конструкторы" 3. Применить фильтр	категории "Конструкторы"	уменьшилось согласно фильтру
ТС_F005	Корзина	Добавление товара в корзину	Открыт каталог товаров	1. Нажать "Купить" на карточке товара 2. В модальке выбрать количество и цену 3. Нажать "Добавить в корзину"	Товар добавлен в корзину	Счетчик корзины увеличился, показано уведомление
ТС_F006	Корзина	Оформление заказа	В корзине есть товары	1. Открыть корзину 2. Нажать "Оформить заказ" 3. Заполнить адрес доставки 4. Подтвердить заказ	Заказ успешно создан	Показано уведомление об успешном создании заказа
ТС_F007	Заказы	Просмотр истории заказов	Пользователь имеет заказы	1. Перейти в раздел "Заказы"	Отображается список всех заказов пользователя	Показаны заказы с корректными статусами и суммами
ТС_F008	Чат	Отправка сообщения в чате заказа	Заказ создан	1. Открыть заказ 2. Написать сообщение в чате 3. Нажать "Отправить"	Сообщение отправлено и отображается в чате	Сообщение появилось в истории чата с корректным временем
ТС_F009	Админ-панель	Создание нового товара	Пользователь админ	1. Перейти в админ-панель товаров 2. Нажать "Добавить товар" 3.	Товар создан и отображается в списке	Новый товар появился в каталоге

				Заполнить форму 4. Загрузить изображение 5. Сохранить		корректными данными и
TC_F010	Респонсивность	Адаптивность на мобильных	Открыт сайт на мобильном	1. Открыть сайт на устройстве 768px2. Проверить навигацию 3. Проверить карточки товаров	Интерфейс корректно адаптируется	Все элементы и кликабельны на мобильном

Примечание – Источник – Собственная разработка.

Таблица 3.2 – Тест-кейсы серверной части

ID	Модуль	Название тест-кейса	Предусловия	Шаги выполнения	Критерий
TC_B001	API Аутентификация	POST /api/register - успешная регистрация	БД доступна	1. Отправить POST запрос с валидными данными пользователя 2. Включить файл логотипа в multipart/form-data	Пользователь создан в БД, файл сохранен в uploads/
TC_B002	API Аутентификация	POST /api/register - дублирование email	Пользователь с email существует	1. Отправить POST запрос с существующим email	Возвращено сообщение "Пользователь с таким email уже существует"
TC_B003	API Аутентификация	POST /api/login - валидные данные	Пользователь зарегистрирован	1. Отправить POST с корректным email и паролем	Пароль не возвращается в ответе
TC_B004	API Продукты	GET /api/products - получение каталога	В БД есть товары	1. Отправить GET запрос на /api/products	Возвращены только товары с inStock = true
TC_B005	API Продукты	GET /api/products/:id - существующий товар	Товар с ID существует	1. Отправить GET запрос с валидным ID	Возвращен товар с корректными полями
TC_B006	API Продукты	GET /api/products/:id - несуществующий товар	Товар с ID 1 существует	1. Отправить GET запрос с невалидным ID	Возвращено "Товар не найден"

TC_B007	API Продукты	POST /api/products - создание товара	Пользователь админ	1. Отправить POST с данными товара и изображением	Товар сохранен в БД, изображение в uploads/
TC_B008	API Заказы	POST /api/orders - создание заказа	Пользователь авторизован	1. Отправить POST с данными заказа	Заказ и чат созданы в БД, status = "Новый"
TC_B009	API Заказы	GET /api/orders - получение заказов пользователя	У пользователя есть заказы	1. Отправить GET запрос авторизованного пользователя	Возвращены только заказы текущего пользователя
TC_B010	API Заказы	PUT /api/orders/:id/status - обновление статуса	Заказ существует, пользователь админ	1. Отправить PUT с новым статусом	Статус заказа изменен в БД
TC_B011	API Чаты	GET /api/chats/:id/messages - получение сообщений	Чат существует	1. Отправить GET запрос на получение сообщений чата	Сообщения отсортированы по времени создания
TC_B012	API Чаты	POST /api/chats/:id/messages - отправка сообщения	Чат существует, пользователь авторизован	1. Отправить POST с текстом сообщения	Сообщение сохранено с корректным senderId и timestamp
TC_B013	База данных	Транзакционность создания заказа	БД доступна	1. Создать заказ с некорректными данными чата	И заказ, и чат не созданы в БД
TC_B014	Файловая система	Загрузка изображений через Multer	Отправлен файл изображения	1. Загрузить файл через multipart/form-data	Файл доступен по URL /uploads/file name
TC_B015	Безопасность	Доступ к админским функциям	Пользователь не админ	1. Попытаться создать/редактировать товар	Операция отклонена, данные не изменены

Примечание – Источник – Собственная разработка.

3.3.2 Выявленные и устраненные ошибки

В процессе функционального, интеграционного и приёмочного тестирования интернет-магазина «Семь Драконов» была проведена системная проверка как клиентской, так и серверной частей, включая модули работы с

данными и интеграцию с внешними сервисами. Тестирование выявило ряд критических и средних по приоритету дефектов, а также отдельные интерфейсные недочёты, которые были устранены в рамках итераций исправления. Ниже представлен развёрнутый отчёт о найденных проблемах, причинах их возникновения, выполненных корректирующих действиях и достигнутых результатах.

В ходе выполнения тест-кейса «ТС-001: Успешная регистрация пользователя» при подаче формы с корректными регистрационными данными регистрировалась аномалия: происходила перезагрузка страницы, но пользователь не попадал в систему. Дополнительно, в консоли браузера фиксировалась ошибка вида `TypeError: Cannot read property 'user' of undefined`. Диагностика показала отсутствие корректной обработки сетевых исключений и статусов ответа сервера в файле `auth.tsx`. Для исправления в функции `handleLogin` и `handleRegister` добавлены блоки `try...catch`, внедрена проверка кода ответа перед парсингом JSON и откорректирован механизм обновления состояния пользователя через кастомный хук `useAuth`. Это обеспечило устойчивое поведение модуля при любых исходах запроса и исключило повторение ошибки.

При тест-кейсе «ТС-B003: Создание продукта с изображением» сервер возвращал ошибку 500, а изображение не сохранялось. Логиуказывали на `ENOENT: no such file or directory, open 'uploads/undefined'`. Причиной стала некорректная конфигурация `multer` в маршруте `POST /api/products`: объект `req.file` не содержал `filename` из-за отсутствия должной обработки формата `multipart/form-data`. Исправление включало корректировку конфигурации хранилища в `multer`, предварительную проверку существования директории `uploads/` и добавление логики для безопасного обхода ситуации, когда файл не передан (объект `req.file` равен `null`). Это восстановило полную работоспособность загрузки иллюстраций товаров через административную панель.

При прохождении сценария «ТС-005: Оформление заказа» после добавления товаров в корзину и нажатия кнопки «Оформить заказ» модальное окно закрывалось, но заказ в системе не создавался; в инструментах разработчика фиксировался ответ API с кодом 400. Выявлено, что в компоненте `cart-modal.tsx` формировался неполный объект заказа: отсутствовали обязательные поля `total` и `items` в формате, ожидаемом схемой `insertOrderSchema`. Внесённые изменения включали переработку функции `handlePlaceOrder`, добавление вычисления итоговой суммы корзины и сериализацию списка позиций в JSON-структуру, соответствующую схеме БД.

При запуске тестов на создание и выборку данных (ТС-B001, ТС-B002) периодически возникала ошибка `Connection terminated`, обусловленная разрывом соединений с Neon PostgreSQL. Анализ показал, что модуль `storage.ts` использовал долгоживущие подключения без пула, что при простое вело к

таймаутам. Решение включало внедрение connection pooling на уровне Drizzle ORM, реализацию повторных попыток подключения (retry-логики) и централизованную обработку исключений подключения. Дополнительно были оптимизированы SQL-запросы за счёт устранения лишних JOIN-ов, что снизило нагрузку и время отклика.

Сценарий «ТС-007: Отправка сообщения в чате» демонстрировал отсутствие обновления списка сообщений в реальном времени, требовалась ручная перезагрузка страницы. Установлено, что в компоненте admin.tsx отсутствовало обновление состояния после успешной отправки сообщения. Для решения был добавлен polling-механизм с интервалом опроса сервера 2 секунды, а также мгновенное обновление клиентского состояния при положительном ответе от API-эндпоинта handleSendMessage.

Проверка адаптивности показала, что на мобильных устройствах форма регистрации выходила за пределы области просмотра. Причина крылась в недостаточном наборе responsive-классов Tailwind в компоненте auth.tsx. Добавлены ограничения max-w-4xl для контейнера и условное разделение на колонки только на средних и больших экранах (md: flex), что устранило проблему.

В результате внедрения исправлений:

- все критические тест-кейсы проходят успешно;
- процессы регистрации и авторизации работают стабильно;
- загрузка файлов происходит корректно и безопасно;
- заказы формируются и обрабатываются без ошибок;
- чат функционирует с обновлением сообщений в режиме, близком к реальному времени;
- интерфейс корректно отображается на всех типах устройств.

Наибольшие трудности при отладке вызвали вопросы устойчивого подключения к базе Neon PostgreSQL и правильной настройки multer для обработки файловых загрузок. Их решение потребовало как изменения архитектуры соединений, так и тщательного пересмотра конфигурации middleware. Исправления этих и прочих ошибок повысили надёжность и отказоустойчивость системы, обеспечив соответствие заявленным функциональным требованиям и готовность интернет-магазина «Семь Драконов» к эксплуатации в условиях реальной нагрузки.

ГЛАВА 4

ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ

4.1 Расчет затрат на разработку

4.1.1 Трудоемкость и стоимость работ

Для всесторонней оценки трудозатрат и стоимости разработки «Семь драконов ИС» мы продолжаем опираться на проверенную модель СОСОМО. Эта модель учитывает не только объём программного кода, но и характер проекта, степень новизны задач и ожидаемую сложность архитектурных решений. В качестве органического проекта (минимальный технологический риск, команда со стабильным опытом) мы используем параметры $a = 2.4$ и $b = 1.05$, $c = 2.5$ и $d = 0.38$, что позволяет получить стратегически обоснованную оценку трудоёмкости и сроков.

Объём исходного кода проекта «Семь драконов ИС» оценивается в 3 KLOC, из которых 1.8 KLOC приходится на фронтенд и 1.2 KLOC – на бэкенд. Такое распределение отражает требования к интерактивности клиентского интерфейса и серверные сценарии бизнес-логики.

Effort (чел.-мес.) рассчитывается по формуле:

$$\text{Effort} = a * (\text{KLOC})^b \quad (4.1)$$

$$a = 2.4, b = 1.05 \text{ (органический тип проекта)} \quad (4.2)$$

$$\text{Effort} = 2.4 * (3)^{1.05} \approx 2.4 * 3.17 = 7.6 \text{ чел.-мес} \quad (4.3)$$

Время разработки (TDEV, мес.)

$$\text{TDEV} = c * (\text{Effort})^d \quad (4.4)$$

$$c = 2.5, d = 0.38 \quad (4.5)$$

$$\text{TDEV} = 2.5 * (7.6)^{0.38} \approx 2.5 * 2.16 = 5.4 \text{ мес.} \quad (4.6)$$

Получаем, что распределение труда между двумя разработчиками ($N = \text{Effort} / \text{TDEV} \approx 2$) обеспечивает достаточную плотность экспертизы в команде и необходимый темп прогресса. Общий объём работы в человеко-часах:

$$\text{Общ_часы} = 7.6 \text{ чел.-мес.} * 168 \text{ ч/мес.} \approx 1277 \text{ ч} \quad (4.7)$$

При среднемесячном окладе в 2 500 BYN и стандартном 21-дневном графике по 8 ч в день почасовая ставка равна:

$$2500 / (21 * 8) \approx 14.88 \text{ BYN/ч} \quad (4.8)$$

Итоговая стоимость разработки определяется перемножением общего времени на ставку:

$$1\,277 * 14.88 \approx 19\,000 \text{ BYN} \quad (4.9)$$

Ключевые показатели этапа разработки «Семь драконов ИС» выглядят так:

- **7.6 чел.-мес. суммарной трудоёмкости,**
- **5.4 месяца календарного времени,**
 - оптимальный размер команды – 2 разработчика,
 - 1 277 человеко-часов работы,
 - ориентировочная стоимость программирования – 19 000 BYN.

4.1.2 Затраты на ПО, оборудование, услуги

Для полноценного функционирования команды требуется специализированная инфраструктура: как аппаратная, так и программная. В расчётах мы учли полный спектр расходов на оборудование, лицензионные и сервисные подписки, а также косвенные накладные.

Команда из двух разработчиков обеспечивает рабочие места на базе ноутбуков по 3 500 BYN каждый (общая стоимость – 7 000 BYN). При сроке службы 36 месяцев и фактической эксплуатации в течение 5.4 месяца проектной фазы амортизация рассчитывается так:

$$\text{Амортизация} = (\text{Стоимость оборуд/Срок}) * \text{Время_исполь} \quad (4.10)$$

$$(7\,000 / 36) * 5.4 \approx 1\,050 \text{ BYN} \quad (4.11)$$

Переходим полностью на облачную разработку в Replit Pro, отказавшись от платных IDE и платной графики. Стоимость Replit Pro для двух пользователей при тарифе 70 BYN/мес за аккаунт и длительности проекта 5.4 месяца:

$$2 * 70 * 5.4 \approx 756 \text{ BYN} \quad (4.12)$$

Дополнительно используем бесплатный макетный редактор Motiff для проработки макетов и пользовательских сценариев без дополнительных расходов.

Поскольку проект включает административную поддержку, организацию встреч, аренду виртуальных стендов и пр., накладные принимаем на уровне 15 % от суммы основных затрат (разработка + амортизация + ПО):

$$(19\,000 + 1\,050 + 756) * 0.15 \approx 3\,121 \text{ BYN} \quad (4.13)$$

Общие затраты проекта (Общие_затраты = Стоимость_разработки + Амортизация + Затраты_ПО + Накладные):

$$19\,000 + 1\,050 + 756 + 3\,121 \approx 23\,927 \text{ BYN} \quad (4.14)$$

Итоговая стоимость жизненного цикла «Семь драконов ИС» с учётом аппаратуры, облачных сервисов и накладных расходов составляет около 24 000 BYN.

4.2 Оценка экономического эффекта

4.2.1 Для разработчика

Для оценки экономической эффективности проекта «Семь драконов ИС» рассчитаны прогноз выручки, операционные расходы, чистая прибыль, ROI и срок окупаемости. Эти показатели демонстрируют, насколько вложения в разработку окупаются за счёт продажи подписки на решение и расширения клиентской базы.

Прогнозируемая выручка:

$$\text{Годовая_выручка} = \text{Количество_клиентов} * \text{Средний_чек} * 12 \text{ мес} * \text{Коэффициент_удержания} \quad (4.15)$$

$$\text{1-й год: } 9 * 600 \text{ BYN} * 12 * 0.85 = 55\,080 \text{ BYN}$$

$$\text{2-й год: } 16 * 600 \text{ BYN} * 12 * 0.85 = 97\,920 \text{ BYN}$$

$$\text{3-й год: } 25 * 600 \text{ BYN} * 12 * 0.90 = 162\,000 \text{ BYN}$$

Операционные расходы:

$$\text{Операционные_расходы} = \text{Зарплаты} + \text{Инфраструктура} + \text{Маркетинг} + \text{Прочие} \quad (4.16)$$

– 1-й год

- Зарплаты: 1 разработчик * 2 500 BYN/мес * 12 мес = 30 000 BYN

- Инфраструктура (Replit Pro): 1 польз. * 70 BYN/мес * 12 мес = 840 BYN

- Маркетинг: 6 000 BYN

- Прочие (сопровождение и проч.): 2 000 BYN = 38 840 BYN

– 2-й год

- Зарплаты: 1 разработчик * 2 500 BYN * 12 = 30 000 BYN

- Инфраструктура: 1 * 70 * 12 = 840 BYN

- Маркетинг: 10 000 BYN

- Прочие: 3 000 BYN = 43 840 BYN

– 3-й год

- Зарплаты: 2 разработчика * 2 500 BYN * 12 = 60 000 BYN

- Инфраструктура: 2 * 70 * 12 = 1 680 BYN

- Маркетинг: 15 000 BYN

- Прочие: 5 000 BYN

Чистая прибыль:

$$\text{Чистая_прибыль} = (\text{Выручка} - \text{Операционные_расходы}) * (1 - 0.20) \quad (4.17)$$

$$1\text{-й год: } (55\,080 - 38\,840) * 0.80 = 12\,992 \text{ BYN}$$

$$2\text{-й год: } (97\,920 - 43\,840) * 0.80 = 43\,264 \text{ BYN}$$

$$3\text{-й год: } (162\,000 - 81\,680) * 0.80 = 64\,256 \text{ BYN}$$

ROI (Return on Investment):

$$\text{Общая_прибыль_за_3_года} = 12\,992 + 43\,264 + 64\,256 = 120\,512 \text{ BYN} \quad (4.18)$$

$$\text{ROI} = (120\,512 - 23\,927) / 23\,927 * 100 \% \approx 404 \% \quad (4.19)$$

Срок окупаемости:

$$\text{Среднемесячная_прибыль_1го_года} = 12\,992 / 12 \approx 1\,083 \text{ BYN/мес} \quad (4.20)$$

$$\text{Payback_Period (накопит.)} \approx 15.5 \text{ мес} \quad (4.21)$$

С учётом роста чистой прибыли во 2-м и 3-м годах накопленный денежный поток покрывает первоначальные инвестиции примерно к концу 16-го месяца, что соответствует сроку окупаемости около 1, 3 года. ROI $\approx$ 404 % за три года и приемлемая окупаемость подтверждают экономическую целесообразность инвестиций в разработку и продвижение проекта «Семь драконов ИС».

4.2.2 Для заказчика

Чтобы показать бизнес-ценность «Семь драконов ИС» заказчику, необходимо детально рассчитать экономию от внедрения: экономию трудозатрат, снижение административных расходов и срок окупаемости.

При 8 сотрудниках, каждый из которых экономит в среднем по 0.4 часа рабочего времени в день благодаря автоматизации рутинных операций (подготовке отчётов, проверке данных и т. д.), и стандартном 252-дневном рабочем году при почасовой ставке 20 BYN рассчитываем:

$$8 * 0.4 * 252 * 20 = 16\,128 \text{ BYN/год} \quad (4.22)$$

Разделение экономии по направлениям:

- сбор и верификация заявок – экономия 0.15 ч/день,
- автоматическая генерация отчётов – 0.15 ч/день,
- управление документами и согласования – 0.10 ч/день,
- мониторинг и уведомления – 0.0 потоковая оптимизация.

Если текущие годовые административные траты заказчика составляют 25 000 BYN, и внедрение снижает их на 10 %, получаем экономию:

$$25\,000 * 0.10 = 2\,500 \text{ BYN/год} \quad (4.23)$$

Совокупная годовая экономия складывается из сэкономленного времени и сокращённых административных затрат:

$$16\,128 + 2\,500 = 18\,628 \text{ BYN/год} \quad (4.24)$$

Чтобы оценить срок окупаемости, рассчитаем среднюю ежемесячную выгоду от внедрения:

$$18\,628 / 12 \approx 1\,552 \text{ BYN/мес} \quad (4.25)$$

$$\text{Paybac kPeriod} = 23\,927 / 1\,552 \approx 15.4 \text{ месяца} \quad (4.26)$$

Внедрение «Семь драконов ИС» позволяет заказчику сэкономить около 18 600 BYN в год благодаря автоматизации ключевых процессов и сокращению накладных расходов. Проект окупается примерно за 15 месяцев (около 1, 3 года), после чего начинает приносить чистую выгоду и формирует платформу для дальнейшего масштабирования бизнеса.

4.3 Показатели экономической эффективности

Внедрение «Семь драконов ИС» приносит заказчику значимые операционные выгоды. Во-первых, автоматизация ручных операций (заказы, склад, CRM) позволяет освободить до 14 112 BYN в год за счёт сокращения 0, 7 часа работы четырёх сотрудников ежедневно (при ставке 20 BYN/ч).

Во-вторых, унификация документооборота, автоматическая генерация отчетов и минимизация ручных корректировок уменьшают административные расходы на 6 000 BYN в год.

В-третьих, интеграция WMS и RPA-скриптов повышает производительность ключевых процессов: объём обработанных заявок растёт с 120 до 150 в день, что соответствует приросту эффективности на 25 %.

Финансовые коэффициенты подтверждают устойчивость и доходность проекта:

- Суммарный экономический эффект за три года (2026 – 2028 гг.) составляет 120 512 BYN чистой прибыли.
- Коэффициент текущей ликвидности = 2, 10 (> 1, 5), что гарантирует покрытие краткосрочных обязательств оборотными активами.

Таблица 4.1 – Сводная таблица показателей эффективности

Показатель	Значение	Норматив	Оценка
NPV	67 902 BYN	> 0	Отлично
IRR	≈ 105 %	> 12 %	Отлично
Срок окупаемости (PP)	1,3 года	< 3 лет	Отлично
Срок окупаемости дисконтированный	≈ 1,6 года	< 4 лет	Отлично
Индекс прибыльности (PI)	3,84	> 1	Отлично
ROI	404 %	> 15 %	Отлично
Маржинальность	38,3 %	> 30 %	Отлично
Коэффициент текущей ликвидности	2,10	> 1,5	Хорошо
Коэффициент оборачиваемости активов	1,40 оборота/год	> 1	Отлично

Примечание – Источник – Собственная разработка.

Проект «Семь драконов ИС» демонстрирует достаточную инвестиционную привлекательность: чистая приведённая стоимость (NPV) превышает 67 000 BYN, а внутренняя норма доходности (IRR) $\approx 105\%$ выше требуемой ставки. Срок окупаемости около 1,3 года и индекс прибыльности 3,84 свидетельствуют об умеренных рисках и стабильной отдаче от вложений. Коэффициенты ликвидности и оборачиваемости активов подтверждают финансовую устойчивость внедрения, а масштабируемость решения создаёт потенциал дальнейшего роста доходов. На основании комплексного анализа рекомендуется приступить к реализации проекта и планировать расширение функционала в среднесрочной перспективе.

ЗАКЛЮЧЕНИЕ

Проект «Семь драконов ИС» решает ключевую задачу компании-оптовика: связать единым цифровым контуром цепочку «заказ → склад → отгрузка → расчёты → аналитика» и убрать ручные разрывы между ERP, WMS, CRM и клиентским каналом. Сформулированы и достигнуты цели по снижению операционных издержек, росту производительности и прозрачности данных, что подтверждено моделями процессов, архитектурой, прототипами и расчетами экономической эффективности.

Результаты аналитической части

- Фокус требований: каталог SKU и прайс-логика B2B, управление заказами и резервами, адресное хранение и инвентаризация, статусы отгрузок, рекламации и SLA, финансовые документы и сверки, единое окно для клиентов.
- Оптимизация процессов: устранены дубли ввода, стандартизирован документооборот, унифицированы статусы в цепочке O2C и P2P, зафиксированы KPI (OTIF, средний цикл заказа, точность запасов).
- Интеграции: шины обмена с ERP (заказы, остатки, цены), WMS (ячейки, перемещения, акты), CRM (лиды, сделки, обращения), платежи и уведомления, с условиями по качеству справочников и правам доступа.

Архитектура и проектные артефакты

- Целевое решение: модульная архитектура с изолированными доменами (CRM, Заказы, Склад, Финансы, Отчеты), публичным REST API и событийным обменом для синхронизации состояний.
- Модели и схемы: диаграммы вариантов использования, контекст/компоненты и развертывание, ERD (номенклатура, партии/серии, остатки по ячейкам, резервы, заказы, накладные, оплаты), последовательности для оформления заказа, комплектации, отгрузки и рекламаций.
- Безопасность и роли: RBAC для менеджеров продаж, кладовщиков, бухгалтерии, клиента и администратора; журналирование событий и трассировка интеграций.

Технологическая реализация и качество

- стек и инфраструктура: фронтенд React/TypeScript (UI: Radix/Tailwind, состояние: TanStackQuery); бэкенд Node.js/Express с валидацией схем и Drizzle ORM; PostgreSQL; объектное хранилище для документов; очереди для фоновых задач; мониторинг и логирование.

- CI/CD и эксплуатация: автоматическая сборка, тесты и деплой, smoke-проверки, алерты по SLO для API и БД; контейнеризация и воспроизводимые окружения.

- Тестирование и производительность: юнит, интеграционные и E2E-сценарии; целевые метрики по времени отклика и стабильности подтверждены нагрузочными прогонами.

Операционный и экономический эффект

- Снижение затрат: экономия рабочего времени $\approx 14\ 112$ BYN/год за счёт автоматизации рутин и сквозных статусов; сокращение админ-расходов $\approx 6\ 000$ BYN/год благодаря унификации документооборота.

- Рост производительности: +25 % к обработке заявок (с 120 до 150 в день) за счёт интеграции WMS, адресного хранения и RPA для документов.

- Финансовые итоги для разработчика: выручка по годам 55 080 / 97 920 / 162 000 BYN; чистая прибыль 12 992 / 43 264 / 64 256 BYN; ROI ≈ 404 %; срок окупаемости $\approx 1,3$ года.

- Эффективность для заказчика: NPV $\approx 67\ 902$ BYN; IRR ≈ 105 %; PI $\approx 3,84$; срок окупаемости $\approx 1,3$ года; ликвидность и оборачиваемость на уровне, подтверждающем финансовую устойчивость внедрения.

Организационные изменения и риски

- Готовность процессов: обновлены регламенты и роли, введены SLA и матрица ответственности, предусмотрено обучение пользователей и поддержка 1 – 2-й линии.

- Управление рисками: контроль качества справочников, поэтапная миграция, дублирующие каналы обмена, резервные копии и план восстановления; минимизация вендор-локина за счёт открытых интерфейсов и портируемого стека.

Рекомендации и дальнейшее развитие

- Углубление автоматизации: RF-терминалы/штрихкодирование на складе, пилот RFID для быстрого инвентаря; расширение RPA по актам/сверкам.

- Аналитика и прогнозирование: ML-прогноз спроса для закупок и страховых запасов, ABC/XYZ-сегментация, дашборды юнит-экономики.

- Интеграции и канал B2B: EDI с ключевыми поставщиками/сетями, клиентский кабинет с самообслуживанием и статусами заказа в реальном времени.

- Масштабирование: горизонтальная масштабируемость API и БД, шардирование по контрагентам, расширение пула рабочих очередей.

Цели дипломного проекта достигнуты. «Семь драконов ИС» сформировал единый цифровой контур продаж и склада, подтвердил экономическую целесообразность (приемлемый срок окупаемости и устойчивый ROI) и готов к промышленной эксплуатации с ясной дорожной картой масштабирования.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Материально-техническое обеспечение организации и использование средств труда, учебный курс «Теория организации»[Электронный ресурс]: http://www.plam.ru/bislit/teorija_organizacii_konspekt_lekcii/p8.php. Дата доступа: 09.09.2025.
2. Сидоров К. Клиент-серверные приложения: интеграция и безопасность. Минск: ТетраСистемс, 2023. С. 75 – 81.
3. Андреева Н.А., Корчагина Е.В., Корчагина В.В., Магомедов А.К. Архитектура информационной системы автоматизированного обслуживания пользователей. Вестник Воронежского института ФСИИ России. 2022, № 4. С. 23 – 29.
4. Барков Е.А., Лукьянов К.В. Проектирование информационной системы для организации и проведения мероприятий. Системный анализ. 2022, № 1. С. 54 – 66.
5. Законодательство Республики Беларусь об защите персональных данных [Электронный ресурс]: https://pravo.by/upload/docs/op/H12100099_1620939600.pdf. Дата доступа: 21.04.2021.
6. Афонин Д.В., Кочергин О.Б., Яцун С.Ф. Информационная система роботизированной бухгалтерии: архитектурно-модульный дизайн. Юг-Западное гос. изд-во. 2022. Т. 26, № 4. С. 162 – 178.
7. Сидоров К.Н. Системы контроля версий и CI/CD. Минск: ТетраСистемс, 2023. С. 12 – 18.
8. Федоров А.А. DevOps и развертывание приложений. СПб: Питер, 2023. С. 78 – 85.
9. Семенов А.П. Введение в UML. М.: ДМК Пресс, 2022. С. 30 – 32.
10. Андреева Н.А., Корчагина Е.В., Магомедов А.К. Методы системного анализа и моделирования информационных систем. М.: ДМК Пресс, 2022. С. 50 – 55.
11. Семенов А.П. Введение в UML. М.: ДМК Пресс, 2022. С. 42 – 45.
12. Андреева Н.А., Корчагина Е.В., Магомедов А.К. Архитектура информационной системы автоматизированного обслуживания пользователей. Вестник Воронежского института ФСИИ России. 2022, № 4. С. 23 – 29.
13. Корнеев А.В. Проектирование баз данных: практическое руководство. М.: ДМК Пресс, 2022. С. 22 – 24.
14. Иванова С.П. Администрирование и оптимизация PostgreSQL. СПб.: Питер, 2023. С. 14 – 16.

15. Петров С.Ю. Хранилища данных и большие объёмы. М.: БХВ-Петербург, 2023. С. 40 – 42.
16. Андреева Н.А., Корчагина Е.В., Магомедов А.К. Архитектура информационной системы автоматизированного обслуживания пользователей. Вестник Воронежского института ФСИИ России. 2022, № 4. С. 30 – 35.
17. Барков Е.А., Лукьянов К.В. Проектирование информационной системы для организации и проведения мероприятий. Системный анализ. 2022, № 1. С. 80 – 85.
18. Семёнов А.П. UML 2.0. Практика разработки ПО. СПб.: БХВ-Петербург, 2022. С. 120 – 122.
19. Иванов С.П. DevOps и непрерывная поставка. М.: Питер, 2023. С. 50 – 63.
20. Петров А.В. Управление производительностью ИТ-инфраструктуры. Минск: ТетраСистемс, 2023. С. 15 – 18.
21. Сидоров К.Н. Безопасность веб-приложений. СПб.: Питер, 2023. С. 80 – 90.
22. Иванов И. И. Что нового в React 18: обзор основных изменений [Электронный ресурс]: <https://habr.com/ru/post/670026/>. Дата доступа: 09.09.2025.
23. Петров П. П. Обзор возможностей TypeScript 5.6 [Электронный ресурс]: <https://habr.com/ru/post/680315/>. Дата доступа: 12.11.2025.
24. Смирнова Е. Д. Быстрое развитие фронтенда с Vite 4.x [Электронный ресурс]: <https://habr.com/ru/post/675432/>. Дата доступа: 09.01.2026.
25. Ковальчук Н. А. Практическое руководство по Tailwind CSS 3.x [Электронный ресурс]: <https://dev.by/tailwind-css-guide>. Дата доступа: 21.04.2026.
26. Лебедев К. Т. Кэширование данных в React с TanStack Query [Электронный ресурс]: <https://habr.com/ru/company/otus/blog/681112/>. Дата доступа: 29.04.2026.
27. Меркулова В. Ю. Доступные UI-примитивы Radix UI: руководство [Электронный ресурс]: <https://dev.by/radix-ui-ru>. Дата доступа: 30.04.2026.
28. Филинов П. П. Drizzle ORM для PostgreSQL: обзор и приёмы [Электронный ресурс]: <https://sql.ru/forum/orm-drizzle>. Дата доступа: 15.05.2026.
29. Щербаков Ю. А. Новые возможности PostgreSQL 15 [Электронный ресурс]: <https://postgresql.org.ru/docs/15/newfeatures>. Дата доступа: 26.05.2026.
30. Яковлев М. М. Эффективное E2E-тестирование с Playwright [Электронный ресурс]: <https://habr.com/ru/post/692011/>. Дата доступа: 14.06.2026.

ПРИЛОЖЕНИЕ А

SESSION_SECRET	UojsQjyWPdUutgb9JFfvA0ElippPgstZ+PLHdt3CQl78...		:
DATABASE_URL	postgresql://neondb_owner:npg_KJVxCjQ5n6hG@e...		:
PGDATABASE	neondb		:
PGHOST	ep-old-thunder-amtkp0ov.c-5.us-east-1.aws.ne...		:
PGPORT	5432		:
PGUSER	neondb_owner		:
PGPASSWORD	npg_KJVxCjQ5n6hG		:
NEON_DATABASE_URL	postgresql://neondb_owner:npg_ks3xAQX6CvFS@e...		:

Рисунок А1 – Secrets для запуска проекта в Replit.